diag180: Counting polyominoes with 180° rotational symmetry by a transfer matrix along diagonals

(production version)

Günter Rote Claude

November 11, 2026

Abstract

This program counts the fixed polyominoes with n cells that are mapped to themselves by a rotation by 180°, for all n up to a given bound N. It builds one half of the polyomino row by row along the *diagonals* of the square grid and keeps, for every intermediate boundary configuration, only the number of ways to reach it. Configurations that cannot be completed with at most N cells are discarded with the help of a lower bound on the number of missing cells. For small n we have checked that this bound is exact. The numbers are one of the ingredients that are needed to extend the count of *free* polyominoes (OEIS A000105) beyond $n = 59$. The program is written in CWEB [8].

Contents

1 Introduction **2**
1.1 Symmetry classes and cases . . . 3
1.2 Diagonal coordinates . . . 3
1.3 The half polyomino . . . 5

2 The state **5**
2.1 Byte keys . . . 6
2.2 The working state . . . 6
2.3 Removing a cell from the frontier . . . 8
2.4 A transition . . . 9

3 Packed keys **9**

4 Counters **12**

5 The tables **13**

6 The lower bound **16**
6.1 Connections on the diagonal lattice . . . 16
6.2 The cactus of gaps . . . 17
6.3 Reaching the sides and the top . . . 17
6.4 Putting the bound together . . . 20

7 The main loop **24**

8 The main program **29**

9 Results and experiments **32**
9.1 Saving space in larger runs . . . 33

10 References **35**

1 Introduction

A *polyomino* is a finite set of cells of the square grid that is connected through common edges. *Fixed* polyominoes are counted up to translation only; *free* polyominoes are counted up to translation, rotation and reflection. By Burnside's lemma, the number of free polyominoes with n cells is

$$\frac{1}{8}\sum_{g\in D_4}\mathrm{fix}_g(n),$$

where $\mathrm{fix}_g(n)$ is the number of fixed polyominoes with n cells that are invariant under the symmetry g of the square, up to translation. For the identity this is the number $A(n)$ of all fixed polyominoes (OEIS A001168 [9]), which is known up to $n = 70$ by the work of Barequet and Ben-Shachar [1, 2]. (The value for $n = 67$ printed in [1] is too large by 2; it is corrected in [2].) The other seven terms count symmetric polyominoes. The number of free polyominoes is currently known up to $n = 59$ (Shirakawa [11]), and in that computation the polyominoes with a 180° rotational symmetry were the bottleneck: they were enumerated one by one.

This program computes $\mathrm{fix}_\rho(n)$ for the rotation ρ by 180° with a transfer-matrix method, in the spirit of Jensen's algorithm [4, 5] and of Knuth's programs POLYNUM and POLYSLAVE [6, 7], but with three special features:

1. We do not build the polyomino P but its "half", the quotient of P under ρ. The connectivity information in the state has to keep track of how each component is related to its own mirror image.

2. The half is built along the diagonals $x + y =$ const of the grid, as in the work of Barequet and Ben-Shachar on fixed polyominoes [1]. Connections between neighbouring components then use *disjoint sets of columns*, which makes it possible to compute the cost of the cheapest completion of a state almost exactly.

3. States that cannot lead to a polyomino with at most N cells are pruned, using this cost (Section 6). In our experiments the growth of the number of states is about 1.25 per cell, compared to roughly 1.43 for counting all fixed polyominoes with the same technique.

Polyominoes with symmetries are easier to count, because they are more restricted and hence fewer in number. We can expect the number of polyominoes that have a rotational symmetry to be roughly $\sqrt{A(n)}$, because the inclusion or exclusion of one cell decides about its mirror cell as well (except possibly for one cell at the centre of the rotation), and thus the number of independent choices is only half as big as without any constraints. Our results (Table 3) confirm this very well: the quotient $\mathrm{fix}_\rho(n)/\sqrt{A(n)}$ lies between 1.60 and 1.64 for every even n with $8 \le n \le 70$, and between 0.78 and 0.87 for every odd n with $9 \le n \le 69$. In both cases it decreases slowly; some values are shown in Table 1. Generally, we can expect the number of polyominoes that have the symmetries in some given symmetry group of order g to be roughly $\sqrt[g]{A(n)}$.

n	$\mathrm{fix}_\rho(n)$	$\sqrt{A(n)}$	quotient	n	$\mathrm{fix}_\rho(n)$	$\sqrt{A(n)}$	quotient
10	310	190	1.624	11	316	367	0.859
20	247196	151541	1.631	21	249283	298301	0.836
30	$2.240 \cdot 10^8$	$1.376 \cdot 10^8$	1.627	31	$2.239 \cdot 10^8$	$2.730 \cdot 10^8$	0.820
40	$2.144 \cdot 10^{11}$	$1.323 \cdot 10^{11}$	1.621	41	$2.130 \cdot 10^{11}$	$2.634 \cdot 10^{11}$	0.809
50	$2.117 \cdot 10^{14}$	$1.312 \cdot 10^{14}$	1.614	51	$2.094 \cdot 10^{14}$	$2.618 \cdot 10^{14}$	0.800
60	$2.132 \cdot 10^{17}$	$1.326 \cdot 10^{17}$	1.608	61	$2.102 \cdot 10^{17}$	$2.652 \cdot 10^{17}$	0.793
70	$2.178 \cdot 10^{20}$	$1.360 \cdot 10^{20}$	1.602	69	$5.353 \cdot 10^{19}$	$6.797 \cdot 10^{19}$	0.788

Table 1: The number of polyominoes with a 180° rotational symmetry compared with the square root of the number $A(n)$ of all fixed polyominoes, for even n (left) and odd n (right).

The program is called in one of the following ways:

```
diag180 [options] N           all cases; prints n and fix(n) for n = 1..N
diag180 [options] N Mx My umax   one case (see below); prints n, #lt, #eq
```

The options are described in Section 8.

¶ The overall structure of the program is conventional.

2 ⟨ Include files 3 ⟩
⟨ Type definitions 18 ⟩
⟨ Global variables 8 ⟩
⟨ Subroutines 9 ⟩
⟨ The main program 80 ⟩

3 ¶ ⟨ Include files 3 ⟩ ≡
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdint.h>
#include <time.h>
```
This code is used in chunk 2.

¶ Some format declarations for the typesetting of this document.

1.1 Symmetry classes and cases

The rotation ρ of a polyomino P that is invariant under it has a centre, and after a translation we may assume that

$$\rho(x, y) = (M_x - x,\ M_y - y) \qquad \text{with } M_x, M_y \in \{0, 1\}.$$

The point (M_x, M_y) is the mirror image of the origin, and the centre of the rotation is the point $(M_x/2, M_y/2)$. There are three types of centres:

- $(M_x, M_y) = (0, 0)$: the centre of a cell. The centre cell is the only cell that is its own image. Hence n is odd if the centre cell belongs to P, and even if it does not (as for a ring around an empty centre). (Type R180C in Shirakawa's notation [11].)
- $(M_x, M_y) = (1, 0)$ or $(0, 1)$: the midpoint of a vertical or horizontal edge. Here n is even. The two cases are mirror images of each other under $x \leftrightarrow y$ and give the same counts; we compute only $(1, 0)$ and count it twice. (Type R180M.)
- $(M_x, M_y) = (1, 1)$: a vertex of the grid; n is even. (Type R180V.)

In the OEIS [9], the three counts are the sequences A390653 (type C), A390654 (type M, for $2n$ cells and for horizontal edges only, which is half of our count) and A390655 (type V, for $2n$ cells), with terms up to 59 and 58 cells from Shirakawa's computation.

These sets are disjoint, and $\mathrm{fix}_\rho(n)$ is the sum of the counts over all centres. For odd n, only the centre type C contributes, and in our experiments the run for $N = 2k + 1$ needed no larger tables than the run for $N = 2k$.

1.2 Diagonal coordinates

We use the coordinates

$$u = x - y, \qquad v = x + y,$$

so that u increases to the right and v increases upwards when the grid is turned by 45°; the cells appear as diamonds in the pictures. A cell (u, v) exists whenever $u + v$ is even, and its four neighbours are $(u \pm 1, v \pm 1)$. The set of cells with a fixed v is a *row* (a diagonal of the original grid); the set of cells with fixed u is a *column*. A neighbour of a cell is always in an adjacent row *and* in an adjacent column.

In these coordinates,

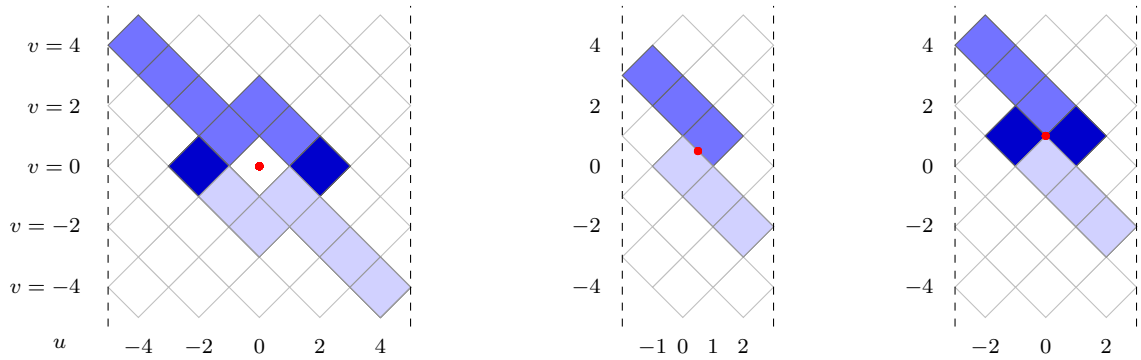
$$\rho(u, v) = (M_u - u,\ M_v - v) \qquad \text{with } M_u = M_x - M_y \text{ and } M_v = M_x + M_y,$$

and the centre is the point $(M_u/2, M_v/2)$. The pairs that correspond to the three cases $(M_x, M_y) = (0, 0)$, $(1, 0)$, $(1, 1)$ are $(M_u, M_v) = (0, 0)$, $(1, 1)$, $(0, 2)$, see Figure 1.

Let $u_{\max}$ be the largest u-coordinate of P. By symmetry the smallest is $u_{\min} = M_u - u_{\max}$. A *case* is a triple $(M_x, M_y, u_{\max})$: we count the symmetric polyominoes that live in the strip $u_{\min} \le u \le u_{\max}$ of

$$L = u_{\max} - u_{\min} + 1 = 2u_{\max} - M_u + 1$$

columns and touch its two sides. (Touching one side implies touching the other.) Inside the program the columns are numbered by their *position* $i = u - u_{\min}$, $0 \le i < L$, and the image of position i is position $L - 1 - i$.



(a) $(M_x, M_y) = (0, 0)$, $(M_u, M_v) = (0, 0)$ (b) $(1, 0)$, $(1, 1)$ (c) $(1, 1)$, $(0, 2)$

Figure 1: Symmetric polyominoes in diagonal coordinates for the three types of centres (red dot). The cells of the half H (Section 1.3) are drawn in blue, their images in light blue, and the cells of the middle row are dark. (a) Centre in a cell, $(M_x, M_y) = (0, 0)$: a polyomino with 14 cells, a 3×3 ring around the empty centre with two arms. The strip has $L = 9$ columns, $u = -4 \,..\, 4$, and the middle row is $v = 0$. (b) Centre on the midpoint of a vertical edge, $(M_x, M_y) = (1, 0)$: 6 cells, $L = 4$, $u = -1 \,..\, 2$. There is no middle row; the twin row $v = 1$ is adjacent to its image, the row $v = 0$. (c) Centre at a vertex, $(M_x, M_y) = (1, 1)$: 8 cells, a 2×2 block with two arms, $L = 5$, $u = -2 \,..\, 2$, middle row $v = 1$.

¶ **Wide and tall polyominoes.** The reflection $y \to -y$ exchanges v and u. It maps polyominoes with a 180° symmetry to polyominoes with a 180° symmetry, and it exchanges the extent in the v-direction (the number of rows, *vspan*) with the extent in the u-direction (*uspan* $= L$). It is therefore sufficient to generate the polyominoes with uspan ≤ vspan:

$$\mathrm{fix}_\rho(n) = \sum_{\text{cases}} \big(2 \cdot \#\{\text{uspan} < \text{vspan}\} + \#\{\text{uspan} = \text{vspan}\}\big).$$

This has two advantages.

First, the strips are narrow. Going from a cell to a neighbour changes both u and v by 1, and hence uspan + vspan ≤ $2n$ for a polyomino with n cells. With uspan ≤ vspan this gives $L \le n$, so that we have to treat the strips with $L \le N$ columns. However, the wide strips among them are cheap: a polyomino that is L columns wide and at least L rows high needs many cells just for reaching the two sides, the top, and the bottom, so that few cells remain for variation, and the pruning leaves few states. The narrow strips are cheap as well, because the number of states is limited by the width. The work is concentrated in the strips of intermediate width, around $L \approx 0.4N$. In the run for $N = 71$, the largest tables occurred for $L = 28$ and $L = 29$ (15 to 20 million states), the strips with $22 \le L \le 34$ took 83 % of the running time, and the strips with $L > 40$ less than 3 %. (The three types of centres C, M, and V took 37 %, 42 %, and 21 % of the time.)

Second, the requirement that P must still grow to a height of at least L rows strengthens the pruning (Section 6.3).

¶ The geometry of the current case is kept in global variables.

```
8  #define MAXL 160      /* maximum number of columns of the strip */
⟨ Global variables 8 ⟩ ≡
   static int NMAX;      /* the bound N on the number of cells */
   static int M_x, M_y;      /* the centre of the rotation is (M_x/2, M_y/2) */
   static int M_u, M_v;      /* the same in diagonal coordinates: M_u = M_x − M_y, M_v = M_x + M_y */
   static int u_min, u_max;      /* the strip: u_min ≤ u ≤ u_max */
   static int L;      /* number of columns of the strip */
   static int E;      /* M_v mod 2 */
   static int NW;      /* length of a vector of counters, NMAX + 1 */
```

See also chunks 13, 22, 29, 49*, 64, 67, and 79*

This code is used in chunk 2.

1.3 The half polyomino

Let H be the set of cells of P with $v > M_v/2$, together with the cells of the *middle row* $v = M_v/2$ if M_v is even. Then $P = H \cup \rho(H)$. The program constructs H row by row, with increasing v, starting at the middle, and within each row from left to right, deciding for each cell whether it belongs to H or not.

Every cell of H outside the middle row stands for two cells of P and contributes 2 to the size n. We call the contribution to n the *weight*. If M_v is even, the middle row is mapped to itself. We do not build it cell by cell; we start with all possible symmetric subsets of the middle row. If M_v is odd, there is no middle row, but the first row $v_0 = (M_v + 1)/2$ of H is adjacent to its own image, the row $v_0 - 1$. We call it the *twin row*; it needs a special treatment of adjacencies.

After k rows beyond the middle have been built, the polyomino extends over $2k + 1$ rows if M_v is even and over $2k$ rows if M_v is odd.

9 ⟨ Subroutines 9 ⟩ ≡

```
static int vspan(int k)
{
  return 2 * k + 1 - E;
}
```

See also chunks 14, 15, 16, 17, 19, 20, 25, 26, 27, 30, 31, 33, 34, 35, 36, 37, 38, 39, 40, 42, 50, 51, 52, 53, 54, 55*, 56, 57, 63*, 65, 66, 68, and 69

This code is used in chunk 2.

2 The state

At any time during the construction, some cells have been decided and the others are still open. The boundary between them is a zigzag line: if the cells of row v have been decided up to position i, the most recently decided cell in each column is in row v for the positions $\le i$ and in row $v - 1$ for the positions $> i$ (always only for the columns of the right parity). These cells form the *frontier*. The place where the frontier steps down from row v to row $v - 1$ is called the *kink*. A frontier cell is dropped from the frontier as soon as both of its upper neighbours have been decided, since nothing can be attached to it any more. (See Figure 2.)

It is useful to imagine that we are simultanously building the polyomino downwards from the center. Thus, there is a lower frontier, at the mirror position of the upper frontier, and if we decide to place a cell at the kink, its mirror will be placed below the lower frontier.

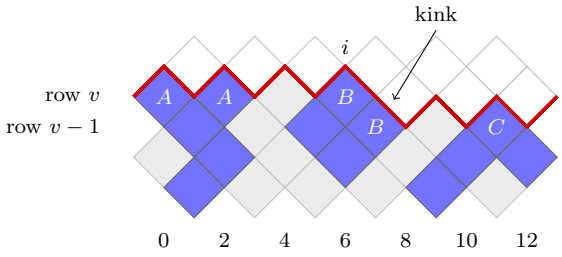


Figure 2: The frontier (thick line) after position $i = 6$ of row v has been decided. Blue cells belong to H, grey cells are decided and empty, white cells are undecided. The frontier consists of the cells of row v at the positions $0, 2, 4, 6$ and the cells of row $v - 1$ at the positions $7, 9, 11$. Five of them are occupied; they form three classes A, B, C with four runs. The frontier steps down at the kink, right of position i.

¶ **Classes and partners.** Consider the part of P that is known so far: the decided cells of H and their images. Its connected components that contain a frontier cell are the *classes*. A *run* is a maximal block of consecutive positions that hold frontier cells. Since the cells of a row occupy only every other position, and the cells in between have already left the frontier, most runs consist of a single cell; only at the kink, the positions i and $i + 1$ can both hold a frontier cell, and these two cells are neighbours. The frontier cells of a class form one or more runs. (In Figure 2, the two cells of class A are two runs, and class B has one run of two cells.) Since the components are separated by the frontier line from the undecided region, two classes cannot *cross*: the pattern $A \dots B \dots A \dots B$ does not occur.

The image $\rho(X)$ of a class X is again a component of the known part. There are three possibilities:

- $\rho(X) = X$. We call X a *self* class. Such a component already connects H to its image.

- $\rho(X) = Y$ is another class. Then X and Y are *partners*.
- $\rho(X)$ has no cell on the frontier. This means that $\rho(X)$ touches the lower frontier. We say that the partner of X is a *ghost*.

When two classes X and Y are joined by a new cell, their images are joined as well, by the image of that cell. This is why the partner relation must be part of the state: if X and Y happen to be partners, the result is a self class; otherwise the new class $X \cup Y$ has the partner $\rho(X) \cup \rho(Y)$.

The polyomino P is connected if and only if, at the end, there is a single class and it is a self class.

¶ In addition, the state contains one bit that tells whether a side of the strip has already been *touched*, i.e., whether the leftmost or rightmost column contains a cell.

For each state we store how many different sets of decided cells lead to it, separately for each weight (number of cells of P so far).

2.1 Byte keys

In its unpacked form, a state is a *byte key* of $\texttt{KL} \leftarrow L + \texttt{MAXC} + 1$ bytes:

- $key[i]$, $0 \le i < L$: the class label of the frontier cell at position i, or 0 if there is none. Labels are numbered $1, 2, \ldots$ in the order of their first appearance from left to right, which makes the key canonical.
- $key[L + j - 1]$, for each class j: its partner, which is another class label, or `PSELF`, or `PGHOST`.
- $key[\texttt{KL} - 1]$: the *touched* bit.

A frontier contains at most $\lceil L/2 \rceil + 1$ cells, hence at most `MAXC` classes.

```
13 #define PSELF  ^FF     /* the class is its own image */
   #define PGHOST  ^FE     /* the image of the class has left the frontier */
⟨ Global variables 8 ⟩ +≡
  static int MAXC;     /* maximum number of classes in a state */
  static int KL;     /* length of a byte key */
```

2.2 The working state

A transition changes only little: a new cell joins at most two classes, and at most two cells leave the frontier. We therefore work directly on a copy of the byte key, with two arrays: $f[i]$ is the label of the frontier cell at position i (or 0), and $par[j]$ is the partner of the class with label j, in the same form as in the byte key. During a transition the labels need not be numbered consecutively, and there may be one label more than in a canonical key. All operations are simple sweeps over the positions.

```
14 #define MAXLAB  (MAXL + 2)     /* bound on the labels in the working arrays */
⟨ Subroutines 9 ⟩ +≡
  static inline int is_class(int p)
  {
    return p ≠ PSELF ∧ p ≠ PGHOST;
  }
  static inline void relabel(uint8_t *f, int from, int to)
  {
    for (int j ← 0; j < L; j++)
      if (f[j] ≡ from) f[j] ← to;
  }
```

¶ **Joining two classes.** When two classes A and B become connected, their images $\rho(A)$ and $\rho(B)$ become connected as well. The cells of B get the label of A, and the partner of the union depends on the partners of A and B:

- If $B = \rho(A)$, the union is its own image: a self class.
- If one of the two is a self class, the union is a self class, and it swallows the partner of the other class if that is on the frontier.
- If both partners are classes on the frontier, these two classes are joined, too, and the result is the partner.
- If only one partner is on the frontier, it is the partner of the union. (It has absorbed the ghost.)
- If both partners are ghosts, so is the partner of the union.

15 ⟨ Subroutines 9 ⟩ +≡

```
static void join(uint8_t *f, uint8_t *par, int a, int b)
{
  if (a ≡ b) return;
  int pa ← par[a], pb ← par[b];
  relabel(f, b, a);
  if (pa ≡ b) par[a] ← PSELF;
  else if (pa ≡ PSELF ∨ pb ≡ PSELF) {
    if (is_class(pa)) relabel(f, pa, a);
    if (is_class(pb)) relabel(f, pb, a);
    par[a] ← PSELF;
  }
  else if (is_class(pa)) {
    if (is_class(pb)) relabel(f, pb, pa);
  }
  else if (is_class(pb)) {
    par[a] ← pb;
    par[pb] ← a;
  }
}
```

¶ In the twin row, we need a variation: the class A becomes connected to the *image* of the class B, and hence $\rho(A)$ becomes connected to B. If $\rho(B)$ is on the frontier (as B itself or as another class), this is an ordinary *join*. Otherwise the image of B is a ghost, which is now attached to A. If $A = B$, this makes A a self class. If A is a self class, it swallows B. If A has a partner on the frontier, that partner swallows B. In the remaining case, both images were ghosts, and now A and B are partners of each other.

16 ⟨ Subroutines 9 ⟩ +≡

```
static void join_image(uint8_t *f, uint8_t *par, int a, int b)
{
  int pa ← par[a], pb ← par[b];
  if (pb ≡ PSELF) join(f, par, a, b);
  else if (is_class(pb)) join(f, par, a, pb);
  else if (a ≡ b) par[a] ← PSELF;
  else if (pa ≡ PSELF) relabel(f, b, a);
  else if (is_class(pa)) relabel(f, b, pa);
  else {
    par[a] ← b;
    par[b] ← a;
  }
}
```

¶ The canonical byte key is obtained by renumbering the classes from left to right. The function returns the number of classes.

17 ⟨ Subroutines 9 ⟩ +≡

```
static int canon(const uint8_t *f, const uint8_t *par, int tch, uint8_t *key)
{
  uint8_t map[MAXLAB], old[MAXLAB];
  int m ← 0;
  memset(map, 0, sizeof (map));
  memset(key, 0, KL);
  for (int i ← 0; i < L; i++) {
    if (¬f[i]) continue;
    if (¬map[f[i]]) {
      map[f[i]] ← ++m;
      old[m] ← f[i];
    }
    key[i] ← map[f[i]];
  }
  for (int j ← 1; j ≤ m; j++) {
    int q ← par[old[j]];
    key[L + j − 1] ← is_class(q) ? map[q] : q;
  }
  key[KL − 1] ← tch;
  return m;
}
```

2.3 Removing a cell from the frontier

When the frontier cell at position i can no longer get new neighbours, it is *retired*. If other frontier cells of the same class remain, nothing happens. Otherwise the class X disappears from the frontier, and there are several cases:

- X is a self class. Then X is a complete symmetric polyomino, and nothing can be added to it. If the frontier is now empty, we have found a polyomino (`COMPLETE`); otherwise the other classes can never be connected to X, and the state is `DEAD`.
- The partner of X is a ghost. Then X and $\rho(X)$ are two different components to which nothing can be added: `DEAD`.
- The partner of X is a class Y on the frontier. From now on, the partner of Y is a ghost.

18 ⟨ Type definitions 18 ⟩ ≡

```
enum {
  OK, DEAD, COMPLETE
};
```

See also chunks 23, 28, and 32

This code is used in chunk 2.

19 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static int retire(uint8_t *f, uint8_t *par, int i)
{
  int lab ← f[i];
  if (¬lab) return OK;
  f[i] ← 0;
  int any ← 0;
  for (int j ← 0; j < L; j++)
    if (f[j]) {
      any ← 1;
      if (f[j] ≡ lab) return OK;
    }
  int p ← par[lab];
```

```
  if (p ≡ PSELF) return any ? DEAD : COMPLETE;
  if (p ≡ PGHOST) return DEAD;
  par[p] ← PGHOST;
  return OK;
}
```

2.4 A transition

The function *step* takes the byte key *sk* of a state (with touched bit *tch*) and decides the cell at position i of the current row: *occ* tells whether the cell belongs to H. The parameter *last* is true for the last cell of the row. The result is DEAD, COMPLETE, or OK; in the last case the new canonical key is stored in *key*, its touched bit in $*ntp$ and its number of classes in $*mp$.

A new cell gets a new class label whose partner is a ghost (its image), and it is joined with the frontier cells at positions $i-1$ and $i+1$. These are its two lower neighbours: at this moment, both positions hold cells of the previous row, see Figure 2. (The class that is given as the first argument of *join* keeps its label.)

After the decision, the cell at position $i-1$ has no undecided neighbours any more and is retired; at the end of a row, the same holds for position $i+1$. (Retiring an empty position does no harm.)

20 ⟨ Subroutines 9 ⟩ +≡

```
static int step(const uint8_t *sk, int tch, int i, int occ, int last, int first_twin, uint8_t *key, int
        *ntp, int *mp)
{
  uint8_t f[MAXL], par[MAXLAB];
  memcpy(f, sk, L);
  memcpy(par + 1, sk + L, MAXC);
  if (occ) {
    int nw ← 1;
    while (nw ≤ MAXC ∧ par[nw])      /* the first unused label */
      nw++;
    par[nw] ← PGHOST;
    f[i] ← nw;
    for (int nb ← i − 1; nb ≤ i + 1; nb += 2)
      if (nb ≥ 0 ∧ nb < L ∧ f[nb]) join(f, par, nw, f[nb]);
    if (first_twin) {
      for (int jj ← L − 2 − i; jj ≤ L − i; jj += 2)
        if (jj ≥ 0 ∧ jj ≤ i ∧ f[jj]) join_image(f, par, nw, f[jj]);
    }
  }
  int status ← OK;
  if (i − 1 ≥ 0) status ← retire(f, par, i − 1);
  if (status ≡ OK ∧ last ∧ i + 1 < L) status ← retire(f, par, i + 1);
  if (status ≠ OK) return status;
  *ntp ← tch ∨ (occ ∧ (i ≡ 0 ∨ i ≡ L − 1));
  *mp ← canon(f, par, *ntp, key);
  return OK;
}
```

¶ **The twin row.** In the twin row $v_0 = (M_v + 1)/2$, which exists if M_v is odd, the lower neighbours of the cell (u, v_0) at position i are the cells $(u \pm 1, v_0 - 1)$. They are the images of the cells of the twin row itself at the positions $L - 1 - (i \pm 1)$, that is, $L - 2 - i$ and $L - i$. If such a position equals i, the new cell is adjacent to its own image and becomes a self class. If it is smaller than i and occupied, the new cell is joined with the *image* of that cell. (Both situations are handled by *join_image*.) If it is larger than i, the adjacency will be found later, when that cell is decided. The parameter *first_twin* is true while the twin row is processed.

3 Packed keys

The byte keys are convenient but wasteful. In the tables, a state is stored as a bit string in KW 64-bit words:

- 3 bits per position, with one of the following six codes 0–5: `C_EMPTY` for an empty position; `C_CONTINUE` if the cell continues the run of its left neighbour; and for the first cell of a run: `C_SINGLETON` if the class consists of this single run, `C_FIRST` for the first run of a class with several runs, `C_MIDDLE` for a middle run, `C_LAST` for the last run. Since classes do not cross, a stack is sufficient to recover the labels, like for a sequence of parentheses.
- 1 bit per class: 0 if the partner is a ghost, 1 otherwise.
- 1 bit: touched.

How long must a key be? Barequet and Ben-Shachar [1, 2] have shown that the number of valid sequences of the five codes other than `C_CONTINUE` for m frontier cells grows like $5^m m^{-3/2}$. Thus, $\log_2 5 \approx 2.32$ bits per frontier cell are necessary, and 3 bits per frontier cell are close to this information-theoretic minimum. However, our key has 3 bits for each of the L positions, and only every other position can hold a frontier cell: the frontier cells of a row occupy the positions of one parity. The only exception is the kink, where a cell of row v and a cell of row $v-1$ are in adjacent positions. (This is also the only place where the code `C_CONTINUE` can occur.) Thus there are only about $m = L/2$ frontier cells, the other positions are always empty, and the key is about twice as long as it would have to be with the same simple encoding, and 2.6 times as long as the minimum. We have kept the straightforward encoding because the keys are only a moderate part of the memory, see Section 9.1, where we also discuss shorter keys.

22 ⟨ Global variables 8 ⟩ +≡
```
static int KW;      /* length of a packed key in 64-bit words */
```

23 ¶ ⟨ Type definitions 18 ⟩ +≡
```
enum {
  C_EMPTY, C_CONTINUE, C_SINGLETON, C_FIRST, C_MIDDLE, C_LAST
};
```

¶ **Why one bit per class is enough.** Call a class *crossing* if it is a self class or has a partner on the frontier. The union of the frontier and its image under ρ is a closed curve around the known part of P (think of a disk), and a crossing class together with its partner connects the frontier with the image of the frontier across this disk. Two such connections cannot cross each other, and ρ reverses the order along the frontier. Consequently, the crossing classes, read from left to right, form one nested chain

$$A_1,\ A_2,\ \ldots,\ A_k,\ (S),\ \rho A_k,\ \ldots,\ \rho A_2,\ \rho A_1,$$

where a self class S is present in the middle if and only if the number of crossing classes is odd. In particular, there is at most one self class. Thus the j-th crossing class from the left is the partner of the j-th crossing class from the right, and it suffices to know *which* classes are crossing. With the option `-C`, the program checks for every state that unpacking the packed key gives back the original byte key. We have run this check for $N \le 30$.

¶ Bit strings are written and read with two small routines. No field has more than 3 bits.

25 ⟨ Subroutines 9 ⟩ +≡
```
static inline void putbits(uint64_t *w, int *pos, uint64_t val, int nb)
{
  int i ← *pos ≫ 6, o ← *pos & 63;
  w[i] |= val ≪ o;
  if (o + nb > 64)  w[i + 1] |= val ≫ (64 − o);
  *pos += nb;
}
static inline int getbits(const uint64_t *w, int *pos, int nb)
{
  int i ← *pos ≫ 6, o ← *pos & 63;
  uint64_t val ← w[i] ≫ o;
  if (o + nb > 64)  val |= w[i + 1] ≪ (64 − o);
  *pos += nb;
  return val & ((1 U ≪ nb) − 1);
}
```

¶ Packing. The first loop determines, for every class, the positions where its first and its last run start.

26 ⟨ Subroutines 9 ⟩ +≡

```
static void pack(const uint8_t *key, uint64_t *w)
{
  int first[MAXL + 2], lastrun[MAXL + 2];
  int pos ← 0, m ← 0;
  memset(w, 0, KW * 8);
  for (int i ← 0; i < L; i++)
    if (key[i] ∧ (i ≡ 0 ∨ ¬key[i − 1])) {
      int l ← key[i];
      if (l > m) {
        m ← l;
        first[l] ← i;
      }
      lastrun[l] ← i;
    }
  for (int i ← 0; i < L; i++) {
    int l ← key[i], code;
    if (¬l) code ← C_EMPTY;
    else if (i > 0 ∧ key[i − 1]) code ← C_CONTINUE;
    else if (first[l] ≡ i) code ← lastrun[l] ≡ i ? C_SINGLETON : C_FIRST;
    else code ← lastrun[l] ≡ i ? C_LAST : C_MIDDLE;
    putbits(w, &pos, code, 3);
  }
  for (int j ← 1; j ≤ m; j++) putbits(w, &pos, key[L + j − 1] ≠ PGHOST, 1);
  putbits(w, &pos, key[KL − 1], 1);
}
```

¶ Unpacking. In the second part, *stack* holds the crossing classes from left to right, and they are matched from the outside to the inside.

27 ⟨ Subroutines 9 ⟩ +≡

```
static void unpack(const uint64_t *w, uint8_t *key)
{
  int stack[MAXL + 2], sp ← 0, m ← 0, pos ← 0;
  memset(key, 0, KL);
  for (int i ← 0; i < L; i++) {
    int code ← getbits(w, &pos, 3);
    switch (code) {
    case C_EMPTY: key[i] ← 0;
      break;
    case C_CONTINUE: key[i] ← key[i − 1];
      break;
    case C_SINGLETON: key[i] ← ++m;
      break;
    case C_FIRST: key[i] ← ++m;
      stack[sp++] ← m;
      break;
    case C_MIDDLE: key[i] ← stack[sp − 1];
      break;
    case C_LAST: key[i] ← stack[−−sp];
      break;
    }
  }
  sp ← 0;      /* crossing classes, left to right */
  for (int j ← 1; j ≤ m; j++)
    if (getbits(w, &pos, 1)) stack[sp++] ← j;
    else key[L + j − 1] ← PGHOST;
```

```
  for (int a ← 0, b ← sp − 1; a ≤ b; a++, b−−) {
    key[L + stack[a] − 1] ← a ≡ b ? PSELF : stack[b];
    key[L + stack[b] − 1] ← a ≡ b ? PSELF : stack[a];
  }
  key[KL − 1] ← getbits(w, &pos, 1);
}
```

4 Counters

The numbers that we count exceed 2^{64} at $n = 66$. There are two ways to deal with this. By default, the counters are unsigned 64-bit integers, and the arithmetic is modulo 2^{64}, or modulo a number $M < 2^{64}$ given by the option `-m`. The true values can then be recovered by the Chinese remainder theorem from two runs with different moduli, which should be relatively prime, for example 2^{64} and $2^{61} - 1$. (The script `tools/crt180.py` does this.) A third possibility, counters with fewer bits and more moduli, is discussed in Section 9.1. Alternatively, if the program is compiled with `-DCNT128`, the counters have 128 bits, and a single run gives the exact values. The records become twice as large, but for our runs with n up to 70, memory is not a bottleneck.

28 ⟨ Type definitions 18 ⟩ +≡

```
#ifdef CNT128
  typedef unsigned __int128 cnt_t;      /* make CNT128=1: exact to 2^128 */
#define CNT_BITS "128"
#else
  typedef uint64_t cnt_t;
#define CNT_BITS "64"
#endif
```

29 ¶ ⟨ Global variables 8 ⟩ +≡

```
  static uint64_t MOD ← 0;      /* the modulus M; 0 means no reduction */
```

30 ¶ ⟨ Subroutines 9 ⟩ +≡

```
  static inline cnt_t addm(cnt_t a, cnt_t b)
  {
    if (¬MOD) return a + b;
    cnt_t s ← a + b;
    if (s < a ∨ s ≥ MOD) s −= MOD;
    return s;
  }
```

¶ There is no `printf` format for 128-bit numbers. The function *cs* converts a counter to a decimal string. It uses eight buffers in rotation, so that several results can be used in the same call of `printf`.

31 ⟨ Subroutines 9 ⟩ +≡

```
  static const char *cs(cnt_t x)
  {
    static char buf[8][44];
    static int r;
    char *e ← buf[r ← (r + 1) & 7] + 43, *p ← e;
    *p ← 0;
    do *−−p ← '0' + (int)(x % 10), x /= 10; while (x);
    return p;
  }
```

5 The tables

The states of one stage of the computation are stored in a hash table with open addressing (linear probing). The states are numbered $0, 1, \ldots, n-1$ in the order of their creation; *keys* contains their packed keys, and the hash table *hash* contains the state numbers plus 1.

The counters of a state form a linked list of *records*, one record per weight for which the count is nonzero. This is much more compact than a full vector of $N+1$ counters, because the weights of a state are confined to a narrow range: the pruning removes all weights that are too large, and at the peak there are only about two records per state.

32 ⟨ Type definitions 18 ⟩ +≡

```
typedef struct {
  cnt_t c;
  uint32_t next;      /* index + 1, 0 = end */
  uint8_t wt;
} rec_t;
typedef struct {
  size_t n, cap, hmask;
  uint64_t *keys;
  uint32_t *head;     /* first record + 1 */
  rec_t *rec;
  size_t nrec, rcap;
  uint32_t *hash;     /* index + 1, 0 = empty */
} table_t;
```

33 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static void *xrealloc(void *p, size_t s)
{
  p ← realloc(p, s);
  if (¬p) {
    fprintf(stderr, "out␣of␣memory\n");
    exit(1);
  }
  return p;
}
```

34 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static void tab_init(table_t *t, size_t cap)
{
  t↣n ← 0;
  t↣cap ← cap;
  size_t h ← 16;
  while (h < 2 * cap) h ≪= 1;
  t↣hmask ← h − 1;
  t↣keys ← xrealloc(0, cap * KW * 8);
  t↣head ← xrealloc(0, cap * 4);
  t↣hash ← calloc(h, sizeof(uint32_t));
  t↣rcap ← 2 * cap;
  t↣nrec ← 0;
  t↣rec ← xrealloc(0, t↣rcap * sizeof(rec_t));
}
static void tab_free(table_t *t)
{
  free(t↣keys);
  free(t↣head);
  free(t↣rec);
  free(t↣hash);
}
```

```
static size_t tab_bytes(const table_t *t)
{
  return t->n * (KW * 8 + 4) + t->nrec * sizeof(rec_t) + (t->hmask + 1) * 4;
}
```

¶ The hash function mixes all words of the key.

35 ⟨ Subroutines 9 ⟩ +≡

```
static inline uint64_t hkey(const uint64_t *k)
{
  uint64_t h ← #9E3779B97F4A7C15ULL;
  for (int i ← 0; i < KW; i++) {
    h ⊕= k[i];
    h *= #BF58476D1CE4E5B9ULL;
    h ⊕= h ≫ 31;
  }
  return h;
}
```

¶ When the table is full, the capacity is doubled and all states are rehashed.

36 ⟨ Subroutines 9 ⟩ +≡

```
static void tab_grow(table_t *t)
{
  size_t cap ← t->cap * 2;
  t->keys ← xrealloc(t->keys, cap * KW * 8);
  t->head ← xrealloc(t->head, cap * 4);
  size_t h ← (t->hmask + 1) * 2;
  free(t->hash);
  t->hash ← calloc(h, sizeof(uint32_t));
  if (¬t->hash) {
    fprintf(stderr, "out␣of␣memory\n");
    exit(1);
  }
  t->hmask ← h − 1;
  t->cap ← cap;
  for (size_t i ← 0; i < t->n; i++) {
    size_t p ← hkey(t->keys + i * KW) & t->hmask;
    while (t->hash[p]) p ← (p + 1) & t->hmask;
    t->hash[p] ← i + 1;
  }
}
static void tab_clear(table_t *t)
{
  t->n ← 0;
  t->nrec ← 0;
  memset(t->hash, 0, (t->hmask + 1) * sizeof(uint32_t));
}
```

¶ The function *tab_get* returns the number of the state with a given byte key. If the state is not yet in the table, it is created, with an empty list of records.

37 ⟨ Subroutines 9 ⟩ +≡

```
static size_t tab_get(table_t *t, const uint8_t *key)
{
  uint64_t w[12];
```

```
  pack(key, w);
  if (CHECK) {
    uint8_t k2[2 * MAXL + 4];
    unpack(w, k2);
    if (memcmp(k2, key, KL)) {
      fprintf(stderr, "pack/unpack␣mismatch\n");
      exit(2);
    }
  }
  size_t p ← hkey(w) & t↦hmask;
  while (t↦hash[p]) {
    size_t i ← t↦hash[p] − 1;
    if (¬memcmp(t↦keys + i * KW, w, KW * 8)) return i;
    p ← (p + 1) & t↦hmask;
  }
  if (t↦n ≡ t↦cap) {
    tab_grow(t);
    p ← hkey(w) & t↦hmask;
    while (t↦hash[p]) p ← (p + 1) & t↦hmask;
  }
  size_t i ← t↦n++;
  t↦hash[p] ← i + 1;
  memcpy(t↦keys + i * KW, w, KW * 8);
  t↦head[i] ← 0;
  return i;
}
```

¶ Adding c to the counter for weight wt of state si.

38 ⟨ Subroutines 9 ⟩ +≡

```
static void tab_add(table_t *t, size_t si, int wt, cnt_t c)
{
  for (uint32_t r ← t↦head[si]; r; r ← t↦rec[r − 1].next)
    if (t↦rec[r − 1].wt ≡ wt) {
      t↦rec[r − 1].c ← addm(t↦rec[r − 1].c, c);
      return;
    }
  if (t↦nrec ≡ t↦rcap) {
    t↦rcap *= 2;
    t↦rec ← xrealloc(t↦rec, t↦rcap * sizeof(rec_t));
  }
  rec_t *r ← &t↦rec[t↦nrec++];
  r↦c ← c;
  r↦wt ← wt;
  r↦next ← t↦head[si];
  t↦head[si] ← t↦nrec;
}
```

¶ For processing a state, its counters are copied into a dense vector gf (“generating function”), which must be zero beforehand, and *tab_unload* clears it again without touching the whole vector.

39 ⟨ Subroutines 9 ⟩ +≡

```
static void tab_load(const table_t *t, size_t si, cnt_t *gf)
{
  for (uint32_t r ← t↦head[si]; r; r ← t↦rec[r − 1].next) gf[t↦rec[r − 1].wt] ← t↦rec[r − 1].c;
}
```

```
static void tab_unload(const table_t *t, size_t si, cnt_t *gf)
{
  for (uint32_t r ← t↦head[si]; r; r ← t↦rec[r − 1].next) gf[t↦rec[r − 1].wt] ← 0;
}
```

¶ Inserting into a table with pruning. The counters *gf* of a state, shifted by the weight *add* of the newly added cell, are added to the state *key* of table *t*, but only for the weights that can still lead to a polyomino with at most N cells, given that at least *lb* further units of weight are necessary. If nothing remains, the state is not even created.

40 ⟨ Subroutines 9 ⟩ +≡

```
static void insert_pruned(table_t *t, const uint8_t *key, const cnt_t *gf, int add, int lb)
{
  int lim ← NMAX − lb − add;
  int any ← 0;
  for (int wt ← 0; wt ≤ lim; wt++)
    if (gf[wt]) {
      any ← 1;
      break;
    }
  if (¬any) return;
  size_t si ← tab_get(t, key);
  for (int wt ← 0; wt ≤ lim; wt++)
    if (gf[wt]) tab_add(t, si, wt + add, gf[wt]);
}
```

6 The lower bound

The key to the efficiency of the program is a good lower bound on the number of cells that must still be added to a state before it can become a complete polyomino that is as high as it is wide. A state whose weight plus this bound exceeds N is discarded.

All bounds in this chapter count cells of H; each such cell has weight 2.

¶ First, a state in which every class has a ghost partner can never be completed: whatever is attached to a class X is attached in mirror image to $\rho(X)$, which is out of reach, so X and $\rho(X)$ can never meet. (The empty state, which occurs only in the twin row, is fine.)

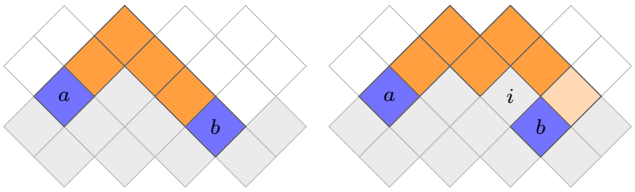
42 ⟨ Subroutines 9 ⟩ +≡

```
static int completion_possible(const uint8_t *key)
{
  int m ← 0;
  for (int j ← 0; j < MAXC; j++) {
    int q ← key[L + j];
    if (¬q) break;
    m++;
    if (q ≠ PGHOST) return 1;
  }
  return m ≡ 0;
}
```

6.1 Connections on the diagonal lattice

Suppose two frontier cells $a < b$ (we identify cells with their positions) in different runs have to be connected, and there are g empty positions between them. A path between them changes the column by ± 1 in every step, hence it needs at least one cell in each of the g columns strictly between a and b. Moreover, a connection between two consecutive runs and a connection between two other consecutive runs lie in disjoint sets of columns, so their costs simply add up. This is the great advantage of diagonal rows over horizontal rows: there, a single new cell can be adjacent to two frontier cells and to its successor in the row.

There is one exception, *at the kink* (Figure 3). Suppose position i has just been decided to be empty, and $b = i + 1$ is a frontier cell in row $v - 1$. The upper neighbours of b are the cell at position i of row v, which is empty, and the cell at position $i + 2$. Hence a path from the left must pass over b and come back to it from the right: it needs an additional cell in the column of b itself (and one in column $i + 2$, which we do not charge, because it might also be used for a connection to the right of b).



(a) $g = 4$ columns, 4 cells (b) at the kink: one more cell

Figure 3: Connecting two frontier cells a and b. (a) One cell per column between them. (b) The cell i left of b has been decided empty, and b is in the lower row: the path must cross the column of b above it. The last cell, drawn lighter, is not charged.

6.2 The cactus of gaps

Which gaps between consecutive runs have to be bridged? We must eventually connect all classes, where a class and its partner count as the same object (if X is connected to Y then ρX is connected to ρY). Consider the graph whose nodes are the classes, with partners identified, and with an edge for each gap between consecutive runs. Because classes do not cross and because of the nested chain structure of the crossing classes, this graph is a *cactus*: its cycles are edge-disjoint, and it can be decomposed as follows.

- *Pockets.* Between two consecutive runs of the same class X there may be classes $C_1, \ldots, C_r$ (at the outermost level inside these two runs). They can only be ghost-partnered, and they appear nowhere else. The gaps X–C_1–$\cdots$–C_r–X form a cycle. To connect everything, all gaps of the cycle except one must be bridged, and the cheapest way is to omit the largest gap. The pockets inside each C_j are independent of this.
- *Outer paths.* At the top level, the classes to the left of A_1 and to the right of ρA_1 hang on a path, and all their gaps must be bridged.
- *Rings.* The gaps from A_j to A_{j+1} on the left together with the gaps from ρA_{j+1} to ρA_j on the right form a cycle, since A_j and ρA_j are the same node. Again the largest gap can be omitted.
- *The middle.* The gaps from A_k over S (if present) to ρA_k form the innermost cycle.

The sum of all gaps minus the largest gap of each cycle is the cost for connecting the cactus.

¶ **The self term.** Connecting the quotient is not enough: in the end, there must be a *self* class. If there is none yet, some class X must be joined to its own image. Closing a ring completely does not achieve this, because the right half of a ring is just the image of its left half. Only a path across the middle, from A_k to ρA_k, creates a self class. Therefore, if there is no self class, the middle cycle must be closed completely, and its largest gap is paid as well.

¶ The function *cycle* handles one cycle with n gaps of sizes g. It returns the sum minus the maximum. The remaining parameters and the calls of *reach* and *sreach* have to do with the height, which is discussed next.

6.3 Reaching the sides and the top

The finished polyomino must touch the sides of the strip, and it must extend to row V^*, the last row for which the height reaches the width L. If no side has been touched yet, the frontier cell closest to a side needs a path of at least as many cells as there are columns between it and the side. In addition, each row above the current row up to row V^* must contain at least one cell. These requirements interact with the connections:

a staircase to the side also climbs one row per cell, and the connecting paths between runs also reach some height.

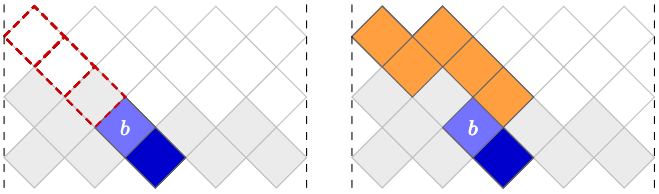
A path of c cells that connects frontier cells in rows r_a and r_b has its highest point at most in row $\lfloor (c + r_a + r_b + 1)/2 \rfloor$: it forms a "tent". The bound adds up the connection costs, determines the largest height h that the connecting paths (or the frontier itself) can reach, and charges one cell for each of the $V^* - h$ rows that are still missing.

There are choices. In a cycle, omitting the largest gap is cheapest, but bridging the largest gap and omitting the second largest one costs only the difference and may reach higher. If several gaps are tied for the largest, we may keep the higher ones for free. The bound takes the minimum over these alternatives. They are collected as *options* (extra cost, peak, score) in the arrays `OPTD`, `OPTP`, `OPTS`.

¶ When no side has been touched yet, the situation is more delicate, because the path to the side may branch off from any cell of the structure, and it helps to reach the top at the same time. From a cell in row h and column j, a staircase that reaches both a side and row V^* needs

$$\max\bigl(V^* - h,\ \min(j,\ L - 1 - j)\bigr)$$

cells (Figure 4). We call the minimum of this quantity over all cells of the structure the *score*; the cells of the structure are the frontier cells and the cells of the connecting paths at their highest possible positions. The alternative is to reach the side and the top separately, at a cost of (distance to the side) + (missing rows).



(a) the bound: 3 cells (b) the truth: 5 cells

Figure 4: The centre is in a cell, $L = 9$, $V^* = 4$. Rows 0 and 1 are complete, and in row 2 the first two cells have been decided empty. (a) The staircase bound for the cell b in row $h = 1$ and column $j = 3$ is $\max(4 - 1, 3) = 3$, but the first cell of the staircase is already known to be empty. (b) Every completion must use the right upper neighbour of b, and 5 cells are necessary. The lookahead at the kink finds this: it tries both possibilities for the next cell.

¶* The variables for the height computation are global, to keep the parameter lists short.

49 ⟨ Global variables 8 ⟩ +≡

```
static int CUR_V, VSTAR, VO;      /* current row v, target row V*, first row v_0 */
static int HO, SO;      /* best height and best score reached for free */
static int NOPT, OPTD[2 * MAXL + 4], OPTP[2 * MAXL + 4], OPTS[2 * MAXL + 4];      /* options */
static int NCL, CLD[MAXL], CLP[MAXL], CLS[MAXL];
    /* the middle cycle: largest gap, its peak and score */
static int DEDGE;      /* distance to the nearest side, if untouched */
static int CUR_TCH;      /* touched bit of the current state */
```

¶ The row of the frontier cell at position j: row `CUR_V` or the row below, depending on the parity.

50 ⟨ Subroutines 9 ⟩ +≡

```
static inline int frow(int j)
{
  return ((u_min + j - CUR_V) % 2 + 2) % 2 ? CUR_V - 1 : CUR_V;
}
```

51 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static inline void reach(int h)
{
  if (h > HO) HO ← h;
}
static inline void sreach(int sc)
{
  if (sc < SO) SO ← sc;
}
```

¶ The cost of reaching both the top row and a side from row h and column j.

52 ⟨ Subroutines 9 ⟩ +≡

```
static inline int both_cost(int h, int j)
{
  int e ← j < L − 1 − j ? j : L − 1 − j;
  int r ← VSTAR − h;
  return e > r ? e : r;
}
```

¶ The best such cost over the cells of a connecting path, each at its highest possible row. The path leads from the frontier cell a in row ra to column e, where it arrives at row $re_$ or lower.

53 ⟨ Subroutines 9 ⟩ +≡

```
static int gap_score(int a, int ra, int e, int re_)
{
  int best ← 1 ≪ 20;
  for (int j ← a + 1; j ≤ e; j++) {
    int t1 ← ra + 1 + (j − a − 1), t2 ← re_ + (e − j);
    int c ← both_cost(t1 < t2 ? t1 : t2, j);
    if (c < best) best ← c;
  }
  return best;
}
```

¶ Now we can give the function *cycle*. Besides returning the sum minus the maximum, it registers the peaks and scores of all gaps that are certainly bridged. If the largest gap $m1$ is unique, bridging it instead of the second largest gap $m2$ is recorded as an option with extra cost $m1 - m2$. If the pointers are nonnull, the largest gap and its peak and score are returned (this is used for the self term).

54 ⟨ Subroutines 9 ⟩ +≡

```
static int cycle(const int *g, const int *pk, const int *sc, int n, int *m1p, int *p1p, int *s1p)
{
  int sum ← 0, m1 ← −1, n1 ← 0, m2 ← 0, p1 ← −1000, s1 ← 1 ≪ 20;
  for (int t ← 0; t < n; t++) {
    sum += g[t];
    if (g[t] > m1) {
      if (m1 ≥ 0) m2 ← m1;
      m1 ← g[t];
      n1 ← 1;
      p1 ← pk[t];
      s1 ← sc[t];
    }
    else if (g[t] ≡ m1) {
      n1++;
      if (pk[t] > p1) p1 ← pk[t];
      if (sc[t] < s1) s1 ← sc[t];
    }
```

```
    else if (g[t] > m2) m2 ← g[t];
  }
  for (int t ← 0; t < n; t++)
    if (n1 ≥ 2 ∨ g[t] ≠ m1) {
      reach(pk[t]);
      sreach(sc[t]);
    }
  if (n1 ≡ 1) {
    OPTD[NOPT] ← m1 − m2;
    OPTS[NOPT] ← s1;
    OPTP[NOPT++] ← p1;
  }
  if (m1p) {
    *m1p ← m1;
    *p1p ← p1;
    *s1p ← s1;
  }
  return sum − m1;
}
```

¶* The function *finish*(h, s) gives the number of cells that are still needed for reaching the top (and the side, if untouched) when the structure reaches height h and score s.

55 ⟨ Subroutines 9 ⟩ +≡

```
static inline int finish(int h, int s)
{
  int top ← VSTAR − h > 0 ? VSTAR − h : 0;
  if (CUR_TCH) return top;
  int sep ← DEDGE + top;      /* edge and top separately */
  return s < sep ? s : sep;
}
```

¶ The function *height_extra* takes the minimum over the options. At most one option is used at a time.

56 ⟨ Subroutines 9 ⟩ +≡

```
static int height_extra(int h, int s)
{
  int ex ← finish(h, s);
  for (int o ← 0; o < NOPT ∧ ex > 0; o++) {
    int e2 ← OPTD[o] + finish(OPTP[o] > h ? OPTP[o] : h, OPTS[o] < s ? OPTS[o] : s);
    if (e2 < ex) ex ← e2;
  }
  return ex;
}
```

6.4 Putting the bound together

The parameter *kink* is the position of the frontier cell right of the kink, if the cell before it has been decided empty, and -1 otherwise.

57 ⟨ Subroutines 9 ⟩ +≡

```
static int cactus_bound(const uint8_t *key, int tch, int kink)
{
  ⟨ Find the runs of the frontier 58 ⟩
  ⟨ Initialise the height computation with the frontier cells 59 ⟩
  int cost ← 0;
```

```
  ⟨ Scan the runs from left to right and pay for the pockets 60 ⟩
  ⟨ Pay for the outer paths, the rings and the middle 61 ⟩
  ⟨ Add the self term and the terms for the side and the top 62 ⟩
  return cost;
}
```

¶ Run j extends from position $rs[j]$ to $re[j]$ and belongs to class $rl[j]$.

58 ⟨ Find the runs of the frontier 58 ⟩ ≡

```
int rs[MAXL], re[MAXL], rl[MAXL], R ← 0;
int lastrun[MAXL + 2];
for (int i ← 0; i < L; i++)
  if (key[i]) {
    if (i ≡ 0 ∨ ¬key[i − 1]) {
      rs[R] ← i;
      rl[R] ← key[i];
      R++;
    }
    re[R − 1] ← i;
    lastrun[key[i]] ← R − 1;
  }
if (¬R) return 0;
```

This code is used in chunk 57.

59 ¶ ⟨ Initialise the height computation with the frontier cells 59 ⟩ ≡

```
HO ← −1000;
SO ← 1 ≪ 20;
NOPT ← 0;
NCL ← 0;
CUR_TCH ← tch;
for (int i ← 0; i < L; i++)
  if (key[i]) {
    reach(frow(i));
    sreach(both_cost(frow(i), i));
  }
```

This code is used in chunk 57.

¶ The pockets are processed with a stack, like nested parentheses. The gaps of the open pockets are collected in a buffer (gb, with peaks pb and scores qb); sl and sb give the class of each open pocket and the start of its gaps in the buffer. Gaps at the top level are stored separately: $topgap[t]$ is the gap after the t-th top-level class $toplab[t]$. When a run of the class on top of the stack is found, the pocket is closed and paid for.

60 ⟨ Scan the runs from left to right and pay for the pockets 60 ⟩ ≡

```
    /* open pockets: class and start of its gaps in the gap buffer */
int sl[MAXL], sb[MAXL], sp ← 0;
int gb[MAXL], pb[MAXL], qb[MAXL], nb ← 0;
int toplab[MAXL], topgap[MAXL], toppk[MAXL], topsc[MAXL], nt ← 0;
for (int j ← 0; j < R; j++) {
  int X ← rl[j];
  if (j > 0) {
    int a ← re[j − 1], b ← rs[j];
    int g ← b − a − 1 + (b ≡ kink);      /* at the kink the path ends one column beyond b: g + 2 cells */
    int pk ← (g − 1 + (b ≡ kink) + frow(a) + frow(b) + 2)/2;
    int sc ← tch ? 1 ≪ 20 : gap_score(a, frow(a), b ≡ kink ? b + 1 : b − 1, frow(b) + 1);
```

```
      if (sp) {
        gb[nb] ← g;
        qb[nb] ← sc;
        pb[nb++] ← pk;
      }
      else {
        topgap[nt − 1] ← g;
        topsc[nt − 1] ← sc;
        toppk[nt − 1] ← pk;
      }
    }
    if (sp ∧ sl[sp − 1] ≡ X) {
      sp−−;
      cost += cycle(gb + sb[sp], pb + sb[sp], qb + sb[sp], nb − sb[sp], 0, 0, 0);
      nb ← sb[sp];
    }
    else if (¬sp) toplab[nt++] ← X;
    if (j < lastrun[X]) {
      sl[sp] ← X;
      sb[sp] ← nb;
      sp++;
    }
  }
```

This code is used in chunk 57.

¶ At the top level, cr lists the crossing classes. Ring number i consists of the gaps between $cr[i]$ and $cr[i+1]$ and their counterparts on the right; the last "ring", $i = k - 1$, is the middle.

61 ⟨ Pay for the outer paths, the rings and the middle 61 ⟩ ≡ /* top level */

```
  int cr[MAXL], m ← 0;
  for (int t ← 0; t < nt; t++)
    if (key[L + toplab[t] − 1] ≠ PGHOST) cr[m++] ← t;
  if (m) {
    for (int t ← 0; t < cr[0]; t++) {
      cost += topgap[t];
      reach(toppk[t]);
      sreach(topsc[t]);
    }
    for (int t ← cr[m − 1]; t < nt − 1; t++) {
      cost += topgap[t];
      reach(toppk[t]);
      sreach(topsc[t]);
    }
    int k ← m/2;
    NCL ← 0;
    for (int i ← 0; i < k; i++) {
      int g[2 * MAXL], pk[2 * MAXL], scs[2 * MAXL], n ← 0, m1 , p1 , s1 ;
      int a ← cr[i], b ← (i < k − 1) ? cr[i + 1] : cr[m − 1 − i];
      for (int t ← a; t < b; t++) {
        g[n] ← topgap[t];
        scs[n] ← topsc[t];
        pk[n++] ← toppk[t];
      }
      if (i < k − 1)
        for (int t ← cr[m − 2 − i]; t < cr[m − 1 − i]; t++) {
          g[n] ← topgap[t];
          scs[n] ← topsc[t];
          pk[n++] ← toppk[t];
        }
```

```
      cost += cycle(g, pk, scs, n, &m1, &p1, &s1);
      if (i ≡ k − 1) {      /* only a path across the middle (A_k .. rho A_k) can join some X to rho X: a
            ring's right side is just the image of its left side */
        CLD[NCL] ← m1;
        CLS[NCL] ← s1;
        CLP[NCL++] ← p1;
      }
    }
    if (m & 1) NCL ← 0;      /* self class present */
  }
  else {
    for (int t ← 0; t < nt − 1; t++) {
      cost += topgap[t];
      reach(toppk[t]);
      sreach(topsc[t]);
    }
  }
```

This code is used in chunk 57.

62 ¶* ⟨ Add the self term and the terms for the side and the top 62 ⟩ ≡

```
DEDGE ← 0;
if (¬tch) {
  int d1 ← rs[0], d2 ← L − 1 − re[R − 1];
  DEDGE ← d1 < d2 ? d1 : d2;
}     /* without a self class the middle cycle must be closed completely: pay its largest gap, which then
       also counts for the height */
if (NCL) {
  int ex ← 1 ≪ 20;
  for (int c ← 0; c < NCL; c++) {
    int e2 ← CLD[c] + height_extra(CLP[c] > H0 ? CLP[c] : H0, CLS[c] < S0 ? CLS[c] : S0);
    if (e2 < ex) ex ← e2;
  }
  cost += ex;
}
else cost += height_extra(H0, S0);
```

This code is used in chunk 57.

¶* The function *bound* returns a lower bound on the *weight* that must still be added to a nonempty state, or -1 if the state can never be completed. Here k is the number of the current row, counted from the first row of H, and *kmin* is the smallest number of rows for which the height reaches L. The cactus bound is not valid in the middle of the twin row, because there the adjacencies to the image row are not yet all known.

63 ⟨ Subroutines 9 ⟩ +≡

```
static int bound(const uint8_t *key, int nt, int k, int kmin, int i, int last, int first_twin)
{
  int lb ← kmin > k ? 2 * (kmin − k) : 0;
  if (¬nt) {
    int dist ← L;
    for (int j ← 0; j < L; j++)
      if (key[j]) {
        int dd ← j < L − 1 − j ? j : L − 1 − j;
        if (dd < dist) dist ← dd;
      }
    if (2 * dist > lb) lb ← 2 * dist;
  }
  if (¬first_twin ∨ last) {
    if (¬completion_possible(key)) return −1;
```

```
    int kink ← (¬last ∧ ¬key[i] ∧ i + 1 < L ∧ key[i + 1]) ? i + 1 : −1;
    CUR_V ← V0 + k − 1;
    VSTAR ← V0 + kmin − 1;
    int eb ← 2 ∗ cactus_bound(key, nt, kink);
    if (eb > lb) lb ← eb;
  }
  return lb;
}
```

7 The main loop

The results of a case are collected in two vectors, indexed by n: `LT` for uspan < vspan and `EQ` for uspan = vspan.

64 ⟨ Global variables 8 ⟩ +≡

```
static cnt_t ∗GF;      /∗ dense scratch vector of counters ∗/
static cnt_t ∗LT, ∗EQ;      /∗ results of one case ∗/
static cnt_t ∗RECT;      /∗ results for the current height ∗/
static size_t MAXSTATES, MAXBYTES, MAXRECS, MAXKW;      /∗ statistics ∗/
```

¶ When a polyomino is complete after *ktop* rows, it is counted if it touches the sides and is at least as high as wide.

65 ⟨ Subroutines 9 ⟩ +≡

```
static void record(int ktop, const cnt_t ∗gf, int tch)
{
  int h ← vspan(ktop);
  if (¬tch ∨ h < L) return;
  cnt_t ∗tgt ← h ≡ L ? EQ : LT;
  for (int wt ← 0; wt ≤ NMAX; wt++)
    if (gf[wt]) {
      tgt[wt] ← addm(tgt[wt], gf[wt]);
      RECT[wt] ← addm(RECT[wt], gf[wt]);
    }
}
```

¶ **Bounding rectangles.** The polyominoes of one case with a given number of rows have the same bounding rectangle in diagonal coordinates: L columns wide and *vspan*(*ktop*) rows high. All polyominoes with *ktop* rows are completed while row *ktop* + 1 is processed. At the end of that row, the function *rect_line* writes one line to the log with the number of polyominoes in this rectangle for every n, and clears the vector `RECT`. The line names the type of centre, the width with the range of u, the height with the range of v, and then the nonzero counts in the form `n=`n`:`count. (The counts are modulo M, like everything else, and they are complete only for $n \le N$.) Only rectangles that are at least as high as wide are reported; the caller tests this. (A line without any counts would mean that no polyomino with at most N cells has this bounding rectangle, although some state survived the pruning up to this row.) The numbers for a rectangle that is wider than high are those of the transposed rectangle, with the same type of centre, except that the vertical edge becomes a horizontal edge; and the case $(M_x, M_y) = (1, 0)$ of a vertical edge that we compute stands also for the case $(0, 1)$ of a horizontal edge with the same numbers. The type of centre is not determined by the size of the rectangle alone: if both sides are even, the centre is the midpoint of an edge, but if both sides are odd, the centre can be the centre of a cell or a vertex of the grid.

66 ⟨ Subroutines 9 ⟩ +≡

```
static void rect_line(int ktop)
{
  static const char ∗tyname[3] ← {"R180C␣centre␣in␣a␣cell",
      "R180M␣centre␣on␣the␣midpoint␣of␣a␣vertical␣edge", "R180V␣centre␣at␣a␣vertex"};
  int h ← vspan(ktop);
  int vtop ← E ? (M_v + 1)/2 + ktop − 1 : M_v/2 + ktop;
```

```
  if (VERB ≥ 1) {
    fprintf(stderr, "#rect␣%s␣(Mx=%d␣My=%d):␣width␣%d␣(u=%d..%d)␣x␣height␣%d␣(v=%d..%d):",
        tyname[M_x + M_y], M_x, M_y, L, u_min, u_min + L − 1, h, M_v − vtop, vtop);
    for (int n ← 0; n ≤ NMAX; n++)
      if (RECT[n]) fprintf(stderr, "␣n=%d:%s", n, cs(RECT[n]));
    fprintf(stderr, "\n");
    fflush(stderr);
  }
  memset(RECT, 0, NW ∗ sizeof(cnt_t));
}
```

¶ Progress is reported on the standard error stream, after every row (`-v 1`, the default) or after every cell (`-v 2`). With `-v 1`, a line is also written within a row when more than `PSEC` seconds have passed since the last line. Every line is flushed immediately. The field "done" is the number of polyominoes that the current case has completed so far (modulo M).

67 ⟨ Global variables 8 ⟩ +≡

```
static int VERB ← 1;      /∗ -v: 0 none, 1 per row, 2 per cell ∗/
static double PSEC ← 60;      /∗ -p: within a row, a line at most every PSEC s ∗/
static double T_START, T_LAST;
```

68 ¶ ⟨ Subroutines 9 ⟩ +≡

```
static double wall(void)
{
  struct timespec ts;
  clock_gettime(CLOCK_MONOTONIC, &ts);
  return ts.tv_sec + 1 · 10^−9 ∗ ts.tv_nsec;
}
static void stat_line(const char ∗tag, int k, int v, int ci, int nu, const table_t ∗cur, const table_t
        ∗nxt)
{
  double now ← wall( );
  T_LAST ← now;
  cnt_t done ← 0;
  for (int n ← 0; n ≤ NMAX; n++) done ← addm(done, addm(LT[n], EQ[n]));
  fprintf(stderr,
      "%s␣Mx=%d␣My=%d␣umax=%d␣L=%d␣row␣k=%d␣v=%d␣cell␣%d/%d:␣""%zu␣states,␣%zu␣rec\
      ords,␣%.1f␣MB,␣done␣%s,␣%.1fs␣wall,␣%.1fs␣cpu\n", tag, M_x, M_y, u_max, L, k, v, ci, nu, cur→n,
      cur→nrec, (tab_bytes(cur) + tab_bytes(nxt))/1 · 10^6, cs(done), now − T_START,
      (double) clock( )/CLOCKS_PER_SEC);
  fflush(stderr);
}
```

¶ Here is the function that treats one case.

69 ⟨ Subroutines 9 ⟩ +≡

```
static void count_case(void)
{
  ⟨ Set up the geometry of the case 70 ⟩
  ⟨ Local variables of count_case 71 ⟩
  ⟨ Create the initial states 72 ⟩
  int cells_v[MAXL];
  for (int k ← 1; cur→n; k++) {
    ⟨ Determine the cells of row k 73 ⟩
    for (int ci ← 0; ci < nu; ci++) {
      ⟨ Decide cell ci of the row for all states 74 ⟩
    }
```

```
    ⟨ Merge every state with its mirror image 78 ⟩
    if (VERB ≥ 1) stat_line("#row␣", k, v, nu, nu, cur, nxt);
    if (vspan(k − 1) ≥ L)      /* lower rectangles are not counted in this case */
      rect_line(k − 1);
  }
  tab_free(&A);
  tab_free(&B);
}
```

¶ The strip with a single column is a special case.

70 ⟨ Set up the geometry of the case 70 ⟩ ≡

```
M_v ← M_x + M_y;
M_u ← M_x − M_y;
u_min ← M_u − u_max;
L ← u_max − u_min + 1;
E ← M_v & 1;
memset(LT, 0, NW ∗ sizeof(cnt_t));
memset(EQ, 0, NW ∗ sizeof(cnt_t));
if (L ≡ 1) {
  if (M_x ≡ 0 ∧ M_y ≡ 0) {
    EQ[1] ← addm(EQ[1], 1);
    RECT[1] ← 1;
    rect_line(0);
  }
  return;
}
if (L > MAXL) {
  fprintf(stderr, "strip␣too␣wide\n");
  exit(1);
}
MAXC ← (L + 1)/2 + 1;
KL ← L + MAXC + 1;
KW ← (3 ∗ L + MAXC + 1 + 63)/64;
NW ← NMAX + 1;
int kmin ← 0;
while (vspan(kmin) < L) kmin++;
```

This code is used in chunk 69.

¶ Two tables are used alternately: *cur* for the states before the current cell and *nxt* for the states after it.

71 ⟨ Local variables of *count_case* 71 ⟩ ≡

```
table_t A, B, ∗cur ← &A, ∗nxt ← &B;
tab_init(&A, 1024);
tab_init(&B, 1024);
uint8_t key[MAXL ∗ 2 + 4];
cnt_t one[MAXL ∗ 2 + 4];
int v0;
```

This code is used in chunk 69.

¶ **Initial states.** If M_v is even, the middle row $v_m = M_v/2$ is its own image. We enumerate all nonempty subsets of it that are symmetric under $i \leftrightarrow L - 1 - i$. Every cell is a class of its own; the partner of a cell is its mirror cell, and the centre cell, if present, is a self class. The weight is the number of cells. If M_v is odd, we start with the empty state before the twin row.

72 ⟨ Create the initial states 72 ⟩ ≡

```
if (E ≡ 0) {
  int vm ← M_v/2;
  int half[MAXL], nh ← 0;
```

```
    for (int i ← 0; i < L; i++)
      if (((u_min + i − vm) % 2 + 2) % 2 ≡ 0 ∧ 2 ∗ (u_min + i) ≥ M_u) half[nh++] ← i;
          /∗ all nonempty subsets of half with at most NMAX cells in the row ∗/
    for (uint64_t mask ← 1; mask < (1ULL ≪ nh); mask++) {
      int cells[MAXL], nc ← 0, isc[MAXL];
      memset(isc, 0, sizeof(int) ∗ L);
      for (int b ← 0; b < nh; b++)
        if (mask ≫ b & 1) {
          isc[half[b]] ← 1;
          isc[L − 1 − half[b]] ← 1;
        }
      for (int i ← 0; i < L; i++)
        if (isc[i]) cells[nc++] ← i;
      if (nc > NMAX) continue;
      uint8_t f[MAXL], par[MAXLAB];
      memset(f, 0, L);
      for (int a ← 0; a < nc; a++) f[cells[a]] ← a + 1;
      for (int a ← 0; a < nc; a++) {
        int i ← cells[a], j ← L − 1 − i;
        par[a + 1] ← j ≡ i ? PSELF : f[j];
      }
      int tch ← isc[0] ∨ isc[L − 1];
      canon(f, par, tch, key);
      memset(one, 0, sizeof(cnt_t) ∗ NW);
      one[nc] ← 1;
      insert_pruned(cur, key, one, 0, 0);
    }
    v0 ← vm + 1;
    V0 ← v0;
  }
  else {
    memset(key, 0, KL);
    memset(one, 0, sizeof(cnt_t) ∗ NW);
    one[0] ← 1;
    insert_pruned(cur, key, one, 0, 0);
    v0 ← (M_v + 1)/2;
    V0 ← v0;
  }
```

This code is used in chunk 69.

¶ The cells of row v are at the positions i for which $u_{\min} + i - v$ is even.

73 ⟨ Determine the cells of row k 73 ⟩ ≡

```
  int v ← v0 + k − 1, nu ← 0;
  for (int i ← 0; i < L; i++)
    if (((u_min + i − v) % 2 + 2) % 2 ≡ 0) cells_v[nu++] ← i;
  int first_twin ← (E ≡ 1 ∧ k ≡ 1);
```

This code is used in chunk 69.

74 ¶ ⟨ Decide cell ci of the row for all states 74 ⟩ ≡

```
  int i ← cells_v[ci];
  int last ← ci ≡ nu − 1;
  tab_clear(nxt);
  for (size_t si ← 0; si < cur→n; si++) {
    ⟨ Extend state si by the two choices for cell i 75 ⟩
  }
  ⟨ Swap the tables and update the statistics 77 ⟩
```

This code is used in chunk 69.

¶ The empty state exists only in the twin row. A polyomino that is completed while row k is processed has its top in row $k-1$.

75 ⟨ Extend state *si* by the two choices for cell *i* 75 ⟩ ≡

```
uint8_t sk[MAXL * 2 + 4];
unpack(cur->keys + si * KW, sk);
const cnt_t *gf ← GF;
tab_load(cur, si, GF);
int tch ← sk[KL − 1];
int empty ← 1;
for (int j ← 0; j < L; j++)
  if (sk[j]) {
    empty ← 0;
    break;
  }
for (int occ ← 0; occ ≤ 1; occ++) {
  if (occ ∧ k > 1 ∧ empty) continue;
  int nt, m;
  int status ← step(sk, tch, i, occ, last, first_twin, key, &nt, &m);
  if (status ≡ DEAD) continue;
  if (status ≡ COMPLETE) {
    record(k − 1, gf, tch);
    continue;
  }
  int add ← occ ? 2 : 0;
  if (m ≡ 0) {
    if (first_twin ∧ ¬last) insert_pruned(nxt, key, gf, add, 0);
    continue;
  }
  int lb ← bound(key, nt, k, kmin, i, last, first_twin);
  if (lb < 0) continue;
  ⟨ Sharpen lb by looking one cell ahead 76 ⟩
  insert_pruned(nxt, key, gf, add, lb);
}
tab_unload(cur, si, GF);
```

This code is used in chunk 74.

¶* **Lookahead.** The bound does not know which cells of the current row are already decided to be empty (Figure 4). This matters most at the kink: if the cell i is empty and there is a frontier cell right of it, that cell depends entirely on the next cell *i2* of the row. We therefore try both possibilities for the next cell and take the smaller of the two bounds, including the weight of the cell. A completion in the lookahead counts only if it would be a valid polyomino.

76 ⟨ Sharpen *lb* by looking one cell ahead 76 ⟩ ≡

```
if (¬last ∧ ¬first_twin ∧ ¬key[i] ∧ i + 1 < L ∧ key[i + 1]) {
    /* one step ahead: the next cell is present or not */
  int i2 ← cells_v[ci + 1], last2 ← ci + 1 ≡ nu − 1;
  int la ← 1 ≪ 20;
  uint8_t k2[MAXL * 2 + 4];
  for (int o2 ← 0; o2 ≤ 1 ∧ la; o2++) {
    int nt2, m2;
    int st2 ← step(key, nt, i2, o2, last2, 0, k2, &nt2, &m2);
    if (st2 ≡ DEAD) continue;      /* a completion only counts if it is a valid polyomino */
    if (st2 ≡ COMPLETE ∧ ¬(nt ∧ vspan(k − 1) ≥ L)) continue;
    int val ← 2 * o2;
    if (st2 ≡ OK) {
      if (m2 ≡ 0) continue;
```

```
        int lb2 ← bound(k2, nt2, k, kmin, i2, last2, 0);
        if (lb2 < 0) continue;
        val += lb2;
      }
      if (val < la) la ← val;
    }
    if (la > lb) lb ← la;
  }
```

This code is used in chunk 75.

77 ¶ ⟨ Swap the tables and update the statistics 77 ⟩ ≡

```
table_t *x ← cur;
cur ← nxt;
nxt ← x;
if (cur→n > MAXSTATES) MAXSTATES ← cur→n;
size_t by ← tab_bytes(cur) + tab_bytes(nxt);
if (by > MAXBYTES) MAXBYTES ← by, MAXRECS ← cur→nrec, MAXKW ← KW;
if (¬last ∧ (VERB ≥ 2 ∨ (VERB ≡ 1 ∧ wall( ) − T_LAST ≥ PSEC)))
  stat_line("#cell", k, v, ci + 1, nu, cur, nxt);
```

This code is used in chunk 74.

¶ **Reflection.** If $M_u = 0$, the strip is symmetric under $u \to -u$, and at the end of a row the mirror image of a state is again a valid state with the same number of completions. We merge each state with its mirror image by keeping the lexicographically smaller byte key. This almost halves the number of states for these cases. (The option `-R` disables it.) The positions are reversed, and *canon* then renumbers the classes.

78 ⟨ Merge every state with its mirror image 78 ⟩ ≡

```
if (REFLECT ∧ M_u ≡ 0 ∧ cur→n) {      /* merge each state with its mirror image u → −u */
  tab_clear(nxt);
  uint8_t rkey[MAXL * 2 + 4];
  for (size_t si ← 0; si < cur→n; si++) {
    uint8_t sk[MAXL * 2 + 4];
    unpack(cur→keys + si * KW, sk);
    uint8_t f[MAXL];
    for (int a ← 0; a < L; a++) f[a] ← sk[L − 1 − a];
    canon(f, sk + L − 1, sk[KL − 1], rkey);
    const uint8_t *use ← memcmp(rkey, sk, KL) < 0 ? rkey : sk;
    tab_load(cur, si, GF);
    insert_pruned(nxt, use, GF, 0, 0);
    tab_unload(cur, si, GF);
  }
  table_t *x ← cur;
  cur ← nxt;
  nxt ← x;
}
```

This code is used in chunk 69.

8 The main program

```
-m M   count modulo M (default: modulo 2^64, or 2^128 with CNT128)
-o F   append the results to the file F (default results180.py)
-v V   progress report: 0 none, 1 per row (default), 2 per cell
-p S   with -v 1, report within a row every S seconds (default 60)
-R     do not merge mirror states
-C     check that packing and unpacking of every key are consistent
```

79 ⟨ Global variables 8 ⟩ +≡
```
static int REFLECT ← 1;     /* merge mirror states? */
static const char *RESFILE ← "results180.py";     /* -o */
static int CHECK ← 0;     /* -C */
```

80 ¶ ⟨ The main program 80 ⟩ ≡
```
int main(int argc, char **argv)
{
  ⟨ Parse the command line 81 ⟩
  clock_t t0 ← clock( );
  T_START ← T_LAST ← wall( );
  ⟨ Print the program parameters 82 ⟩
  if (argc − a ≡ 4) {
    ⟨ Count a single case and print the result 83 ⟩
  }
  else {
    ⟨ Count all cases and add them up by type of centre 84 ⟩
    ⟨ Append the results to the results file 85 ⟩
    ⟨ Print the table of results 86 ⟩
  }
  ⟨ Print the final statistics 87 ⟩
  return 0;
}
```
This code is used in chunk 2.

81 ¶* ⟨ Parse the command line 81 ⟩ ≡
```
int a ← 1;
while (a < argc ∧ argv[a][0] ≡ '-') {
  if (¬strcmp(argv[a], "-m") ∧ a + 1 < argc) MOD ← strtoull(argv[++a], 0, 10), a++;
  else if (¬strcmp(argv[a], "-o") ∧ a + 1 < argc) RESFILE ← argv[++a], a++;
  else if (¬strcmp(argv[a], "-C")) CHECK ← 1, a++;
  else if (¬strcmp(argv[a], "-v") ∧ a + 1 < argc) VERB ← atoi(argv[++a]), a++;
  else if (¬strcmp(argv[a], "-p") ∧ a + 1 < argc) PSEC ← atof(argv[++a]), a++;
  else if (¬strcmp(argv[a], "-R")) REFLECT ← 0, a++;
  else break;
}
if (a < argc ∧ argv[a][0] ≡ '-') {
  fprintf(stderr, "%s: unknown option %s\n", argv[0], argv[a]);
  return 1;
}
if (a ≥ argc) {
  fprintf(stderr, "usage: %s [options] N [Mx My umax]\n", argv[0]);
  return 1;
}
NMAX ← atoi(argv[a]);
NW ← NMAX + 1;
LT ← calloc(NW, sizeof(cnt_t));
GF ← calloc(NW, sizeof(cnt_t));
EQ ← calloc(NW, sizeof(cnt_t));
RECT ← calloc(NW, sizeof(cnt_t));
```
This code is used in chunk 80.

¶* Both output streams start with the parameters of the run, beginning with the complete command line (including the name *argv*[0] of the program), so that every log file and every result file is self-describing.

82 ⟨ Print the program parameters 82 ⟩ ≡
```
for (int str ← 0; str < 2; str++) {
  FILE *f ← str ? stdout : stderr;
```

```
  fprintf(f, "# command:");
  for (int q ← 0; q < argc; q++) fprintf(f, " %s", argv[q]);
  fprintf(f, "\n# diag180 built %s %s; N=%d", __DATE__, __TIME__, NMAX);
  if (argc − a ≡ 4)
    fprintf(f, ", single case Mx=%s My=%s umax=%s", argv[a + 1], argv[a + 2], argv[a + 3]);
  if (MOD) fprintf(f, ", modulus %llu\n", (unsigned long long) MOD);
  else fprintf(f, ", modulus 2**"CNT_BITS"\n");
  fprintf(f, "# production version (cactus bound with height terms, looka\
      head at the kink); reflection merge %s; pack check %s\n", REFLECT ? "on" : "off (-R)",
      CHECK ? "on (-C)" : "off");
  fprintf(f, "# output: results file %s, progress -v %d -p %g s\n", RESFILE, VERB, PSEC);
  fprintf(f, "# sizes: counter %zu bytes, count record %zu bytes, key word 64 bits\n",
      sizeof(cnt_t), sizeof(rec_t));
  fflush(f);
}
```

This code is used in chunk 80.

¶ A single case can be run separately, for example for distributing the cases over the nodes of a cluster.

83 ⟨ Count a single case and print the result 83 ⟩ ≡

```
M_x ← atoi(argv[a + 1]), M_y ← atoi(argv[a + 2]), u_max ← atoi(argv[a + 3]);
count_case( );
if (MOD)
  printf("# modulus %llu; n # (uspan<vspan) # (uspan=vspan)\n", (unsigned long long) MOD);
else printf("# modulus 2**"CNT_BITS"; n # (uspan<vspan) # (uspan=vspan)\n");
for (int n ← 1; n ≤ NMAX; n++) printf("%d %s %s\n", n, cs(LT[n]), cs(EQ[n]));
```

This code is used in chunk 80.

¶ The cases are $(M_x, M_y) \in \{(0,0),(1,0),(1,1)\}$ and all $u_{\max}$ for which $L \le N$. The edge-centred case $(1,0)$ is counted twice, since it stands also for its mirror image $(0,1)$.

84 ⟨ Count all cases and add them up by type of centre 84 ⟩ ≡

```
    /* centre types: 0 cell, 1 edge midpoint, 2 vertex */
cnt_t *out ← calloc(3 * NW, sizeof(cnt_t));
for (M_x ← 0; M_x ≤ 1; M_x++)
  for (M_y ← 0; M_y ≤ M_x; M_y++) {
      /* the case (0, 1) is the mirror image of (1, 0): same counts, so we count (1, 0) twice instead */
    int t ← M_x − M_y, ty ← M_x + M_y, twice ← M_x ≠ M_y;
    for (u_max ← t; 2 * u_max − t + 1 ≤ NMAX; u_max++) {
      count_case( );
      for (int n ← 1; n ≤ NMAX; n++) {
        cnt_t *o ← &out[ty * NW + n];
        *o ← addm(*o, addm(LT[n], LT[n]));
        *o ← addm(*o, EQ[n]);
        if (twice) {
          *o ← addm(*o, addm(LT[n], LT[n]));
          *o ← addm(*o, EQ[n]);
        }
      }
    }
  }
```

This code is used in chunk 80.

¶ The results are appended to a file in Python syntax, with lines of the form

```
Results[n, m, 'R180C'] = x
```

where m is the modulus. Runs with different moduli accumulate in the same file, and the final combination (Chinese remaindering, Burnside's lemma) can be done in Python with integers of unlimited size.

85 ⟨ Append the results to the results file 85 ⟩ ≡ /* results file for the final combination (CRT, Burnside) in Python; appended, so runs with different moduli accumulate */

```
FILE *rf ← fopen(RESFILE, "a");
if (¬rf) {
  perror(RESFILE);
  return 1;
}
char mstr[32];
if (MOD) snprintf(mstr, sizeof mstr, "%llu", (unsigned long long) MOD);
else snprintf(mstr, sizeof mstr, "2**"CNT_BITS);
fprintf(rf,
    "# diag180 N=%d modulus %s\ntry:\n    Results\nexcept NameError:\n    Results = {}\n",
    NMAX, mstr);
static const char *tyname[3] ← {"R180C", "R180M", "R180V"};
for (int n ← 1; n ≤ NMAX; n++)
  for (int ty ← 0; ty < 3; ty++)
    fprintf(rf, "Results[%d, %s, '%s'] = %s\n", n, mstr, tyname[ty], cs(out[ty * NW + n]));
fclose(rf);
printf("# modulus %s; results appended to %s\n", mstr, RESFILE);
```

This code is used in chunk 80.

¶ The table of results has a line for each n with the total $\mathrm{fix}_\rho(n)$ and its three parts for the three types of centres.

86 ⟨ Print the table of results 86 ⟩ ≡

```
printf("# n fix(r180) R180C(centre in a cell) R180M(edge midpoint) R180V(vertex)\n");
for (int n ← 1; n ≤ NMAX; n++) {
  cnt_t tot ← addm(addm(out[n], out[NW + n]), out[2 * NW + n]);
  printf("%d %s %s %s %s\n", n, cs(tot), cs(out[n]), cs(out[NW + n]), cs(out[2 * NW + n]));
}
```

This code is used in chunk 80.

87 ¶ ⟨ Print the final statistics 87 ⟩ ≡

```
fprintf(stderr,
    "# peak memory %.1f MB (both tables; %zu count records, key %zu 64-bit words)\n",
    MAXBYTES/1 · 10^6, MAXRECS, MAXKW);
fprintf(stderr, "# max states %zu, %.2fs\n", MAXSTATES, (double)(clock( ) − t0)/CLOCKS_PER_SEC);
```

This code is used in chunk 80.

9 Results and experiments

Correctness. The program has been checked in several ways. The results agree with a direct enumeration of all polyominoes by Redelmeier's algorithm [10] for $n \le 16$, and with an independent Python prototype of the same method for $n \le 36$. Runs with different moduli are consistent. For $n \le 59$, the totals of our run with $N = 60$ agree with the values in the OEIS (sequences A390653–A390655), which were computed by Shirakawa [11] with a completely different method; for $n \le 32$ we have also compared the three types of centres separately. In the full version of the program, different variants of the bound and of the lookahead can be selected, and they lead to the same counts; this is a strong test, since an invalid bound would lose polyominoes.

Tightness of the bound. For all states that arise for $N \le 22$, after every cell, we have computed the true minimum weight of a completion by exhaustive search. The bound, together with the lookahead at the kink, was equal to the true value for every one of the 87,842 states. Thus, at least in this range, no state is stored in vain.

Growth. Table 2 shows the maximum number of states in a table and the running time. The number of states grows by a factor of about 1.25 when N increases by one. The run for $N = 60$ took 66 minutes on a single core and needed 304 MB (before the mirror-image case $(0, 1)$ was eliminated, which saves one third

N	max. states	records	MB	time (s)
16	70	146	0.0	0.01
20	219	409	0.0	0.03
24	550	1,225	0.1	0.16
28	1,417	3,350	0.2	0.62
32	3,674	8,308	0.4	1.72
36	9,793	20,223	1.1	5.34
40	21,191	44,777	2.8	16.32
44	55,629	130,869	7.4	49.63
48	147,608	352,534	21.3	149.08
60	2,137,770	4,689,188	304.1	3991 (other machine, see text)
71	19,724,251	51,445,692	2959.3	49264 (a third machine, see text)

Table 2: Maximum number of states in one table, number of count records at the peak, memory for both tables, and running time on a single core (64-bit counters).

of the time). The production run for $N = 71$ took 13.7 hours on a single core and needed 3 GB. It was done twice, modulo 2^{64} and modulo $2^{61} - 1$, with identical statistics. (The largest value has 68 bits.)

Values. Table 3 lists the results for $n \le 71$. The values for $n \le 59$ (types M and V: $n \le 58$) agree with the OEIS; the others are new. As a further check, we have derived the numbers of *free* polyominoes whose symmetry group is exactly the group generated by the 180° rotation (OEIS A006747 and its three parts A351615, A234008, A351616) from our numbers and from the known counts of the polyominoes with more symmetries. This involves divisions by 4 and by 2, which always came out exact, and the known terms for $n \le 59$ were reproduced.

Odd and even n. The last column of Table 3 does not grow uniformly: the values come in pairs, with $\mathrm{fix}_\rho(2k+1)$ close to $\mathrm{fix}_\rho(2k)$. For even n, all three types of centres contribute, and type M alone accounts for about 80 % of the total; for odd n there is only type C with an occupied centre cell. The quotient $\mathrm{fix}_\rho(2k+1)/\mathrm{fix}_\rho(2k)$ is 1.019 for $2k = 10$ and decreases steadily: it is 1.001 for $2k = 28$, 0.9995 for $2k = 30$, and 0.983 for $2k = 70$. Thus, for all even $n = 2k$ with $30 \le n \le 70$, the values even go *down* from $n = 2k$ to $n = 2k+1$. Over two steps, the growth is very regular: $\mathrm{fix}_\rho(70)/\mathrm{fix}_\rho(68) \approx 4.002$, which is close to the growth $A(70)/A(69) \approx 4.005$ of the number of all fixed polyominoes in *one* step, in accordance with the square-root heuristic of the introduction (see also Table 1).

9.1 Saving space in larger runs

For $N = 71$, the memory requirement of 3 GB was no obstacle, and we have not tried to reduce it. At the peak, a state needs about 66 bytes: 16 bytes for the packed key (two 64-bit words for $L = 29$), 8 bytes for the hash table entry and the list head, and 16 bytes for each of its count records (8 bytes for the counter, 4 for the link, 1 for the weight, 3 wasted by alignment), of which there are 2.6 per state. (The actual memory at the peak was 150 bytes per state of the larger table, because the old and the new table exist at the same time, and because the arrays grow by doubling and are not full.) The number of states grows by a factor of about 1.25 per unit of N, that is, by a factor of 10 when N increases by 10. If space becomes scarce in larger runs, there are several options. We list them roughly in the order of increasing effort.

1. *Distributing the cases.* The cases $(M_x, M_y, u_{\max})$ are independent, and the program can already run a single case. This saves time when several machines are available, but not space: the peak is determined by the one or two worst strips.

2. *Smaller counters.* With 32-bit counters and arithmetic modulo a prime below 2^{32}, a count record can be packed into 8 bytes (32 bits for the counter, 24 for the link, 8 for the weight) instead of 16. The price is a larger number of runs: three moduli are needed for the 68-bit values at $n = 70$, instead of two. These runs are independent and can be done in parallel. A cheaper intermediate step is to store the records of a state contiguously instead of as a linked list, which removes the link.

3. *Shorter keys.* As explained in Section 3, only every other position can hold a frontier cell. Storing 3 bits only for the $\lceil L/2 \rceil + 1$ positions where a frontier cell is possible (the positions of the right parity on both sides of the kink, and the kink) halves this part of the key. For $L = 29$ the key would then fit into one word instead of two. Ranking the states, as in [3], would give the shortest possible keys, with $\log_2 5 \approx 2.32$ bits per frontier cell, but ranking and unranking are slow.

n	centre in a cell (C)	edge midpoint (M)	vertex (V)	$\mathrm{fix}_\rho(n)$
1	1	0	0	1
2	0	2	0	2
3	2	0	0	2
4	0	6	1	7
5	7	0	0	7
6	0	20	4	24
7	24	0	0	24
8	1	70	14	85
9	86	0	0	86
10	6	254	50	310
11	316	0	0	316
12	27	944	186	1,157
13	1,178	0	0	1,178
14	114	3,556	704	4,374
15	4,442	0	0	4,442
16	472	13,520	2,685	16,677
17	16,896	0	0	16,896
18	1,926	51,792	10,312	64,030
19	64,716	0	0	64,716
20	7,789	199,566	39,841	247,196
21	249,283	0	0	249,283
22	31,348	772,640	154,642	958,630
23	964,708	0	0	964,708
24	125,802	3,003,322	602,468	3,731,592
25	3,748,031	0	0	3,748,031
26	503,948	11,713,496	2,354,592	14,572,036
27	14,610,484	0	0	14,610,484
28	2,016,677	45,816,460	9,227,412	57,060,549
29	57,118,440	0	0	57,118,440
30	8,065,830	179,656,144	36,246,300	223,968,274
31	223,859,532	0	0	223,859,532
32	32,251,819	706,012,804	142,671,941	880,936,564
33	879,285,686	0	0	879,285,686
34	128,955,260	2,779,850,708	562,600,898	3,471,406,866
35	3,460,424,846	0	0	3,460,424,846
36	515,657,653	10,964,180,124	2,222,092,230	13,701,930,007
37	13,642,112,667	0	0	13,642,112,667
38	2,062,335,114	43,311,037,828	8,789,217,470	54,162,590,412
39	53,865,266,960	0	0	53,865,266,960
40	8,250,061,654	171,325,633,988	34,809,890,792	214,385,586,434
41	212,982,833,863	0	0	212,982,833,863
42	33,011,955,188	678,563,531,242	138,027,690,188	849,603,176,618
43	843,202,561,450	0	0	843,202,561,450
44	132,132,934,138	2,690,629,691,636	547,891,969,992	3,370,654,595,766
45	3,342,114,787,290	0	0	3,342,114,787,290
46	529,032,164,150	10,679,964,797,364	2,176,951,996,156	13,385,948,957,670
47	13,260,804,930,438	0	0	13,260,804,930,438
48	2,118,783,780,213	42,432,707,438,606	8,657,510,269,463	53,209,001,488,282
49	52,667,141,592,300	0	0	52,667,141,592,300
50	8,488,385,208,118	168,738,323,390,010	34,458,522,549,812	211,685,231,147,940
51	209,361,911,445,176	0	0	209,361,911,445,176
52	34,017,019,523,282	671,551,934,303,384	137,256,376,227,738	842,825,330,054,404
53	832,940,890,785,598	0	0	832,940,890,785,598
54	136,363,468,404,368	2,674,683,627,090,628	547,112,224,619,120	3,358,159,320,114,116
55	3,316,370,384,325,374	0	0	3,316,370,384,325,374
56	546,798,865,890,672	10,660,289,373,493,158	2,182,260,930,530,668	13,389,349,169,914,498
57	13,213,587,492,371,313	0	0	13,213,587,492,371,313
58	2,193,217,495,584,004	42,515,621,761,219,428	8,709,713,766,597,954	53,418,553,023,401,386
59	52,682,486,874,469,758	0	0	52,682,486,874,469,758
60	8,799,475,720,692,402	169,665,022,179,518,808	34,781,678,682,411,048	213,246,176,582,622,258
61	210,174,787,044,287,374	0	0	210,174,787,044,287,374
62	35,314,186,539,986,938	677,459,634,769,025,376	138,973,126,432,505,228	851,746,947,741,517,542
63	838,970,377,476,893,422	0	0	838,970,377,476,893,422
64	141,760,480,848,748,369	2,706,490,511,006,608,570	555,559,273,399,502,783	3,403,810,265,254,859,722
65	3,350,801,724,691,573,155	0	0	3,350,801,724,691,573,155
66	569,207,945,412,562,900	10,818,015,990,722,597,900	2,221,959,821,577,283,358	13,609,183,757,712,444,158
67	13,389,758,925,336,116,348	0	0	13,389,758,925,336,116,348
68	2,286,087,340,672,559,182	43,260,748,052,225,576,624	8,890,707,275,472,466,316	54,437,542,668,370,602,122
69	53,531,052,184,564,034,510	0	0	53,531,052,184,564,034,510
70	9,183,688,164,160,710,980	173,074,897,071,650,017,240	35,589,330,954,315,943,862	217,847,916,190,126,672,082
71	214,109,513,347,417,644,862	0	0	214,109,513,347,417,644,862

Table 3: The number of fixed polyominoes with n cells that are invariant under a rotation by 180°, by the type of centre. The entries for $n \geq 60$ are new.

4. *Separating the structure from the counting.* Knuth's programs POLYNUM and POLYSLAVE [6, 7] split the work into two phases. The first program runs the transfer-matrix method without any counters: it only finds out which states exist and which state leads to which, and it writes this as a sequence of elementary commands (essentially "add counter a to counter b" and "clear counter a") to a file. The second program executes these commands on an array of counters. It needs neither keys nor a hash table, it reads the commands sequentially, and it can be run several times with different moduli, or once with large counters. The first phase, in turn, needs no counters, only the keys, and, in our setting, for each state the range of weights that survive the pruning.

5. *External storage.* The hash table can be replaced by sorting: the successor states of a row are written to files in sorted runs, and the runs are merged, adding up the counters of equal keys. Only sequential access is needed, so that the tables can live on disk, and the main memory holds just the buffers. A simpler variant keeps the hash table for the new row in memory but reads the old row, which is only scanned once, from a file.

6. *Splitting a case.* A case can be divided into independent subproblems according to the set of occupied cells in the middle row (or in the twin row), and the results are added. Each subproblem has fewer states, at the expense of the total running time, because states that are reached in several subproblems are processed several times. We have not tried how much this would save.

10 References

[1] Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited. *Algorithmica*, 88:Article 51, 2026. Preliminary version in: Proc. Symposium on Algorithm Engineering and Experiments (ALENEX), SIAM, 2024, pp. 133–143. doi:10.1007/s00453-026-01392-6.

[2] Gill Barequet and Gil Ben-Shachar. Counting polyominoes, revisited: Corrigendum and addendum. Preprint, October 2026. arXiv:2610.10632.

[3] Gill Barequet, Micha Moffie, Ares Ribó, and Günter Rote. Counting polyominoes on twisted cylinders. *Integers*, 6:Article A22, 2006.

[4] Iwan Jensen. Enumerations of lattice animals and trees. *Journal of Statistical Physics*, 102:865–881, 2001.

[5] Iwan Jensen. Counting polyominoes: a parallel implementation for cluster computing. In *Computational Science — ICCS 2003*, volume 2659 of *Lecture Notes in Computer Science*, pages 203–212. Springer, 2003.

[6] Donald E. Knuth. POLYNUM. CWEB program, 2016. URL: https://cs.stanford.edu/~knuth/programs.html.

[7] Donald E. Knuth. POLYSLAVE. CWEB program, 2024. URL: https://cs.stanford.edu/~knuth/programs.html.

[8] Donald E. Knuth and Silvio Levy. *The CWEB System of Structured Documentation.* Addison-Wesley, 1994.

[9] OEIS Foundation. The On-Line Encyclopedia of Integer Sequences. Sequences A000105, A001168, A006747, and A390653–A390655. URL: https://oeis.org.

[10] D. Hugh Redelmeier. Counting polyominoes: yet another attack. *Discrete Mathematics*, 36:191–203, 1981.

[11] Toshihiro Shirakawa. Enumeration of polyominoes up to size $N = 59$. Preprint, October 2025. arXiv:2510.22446.

Changed Chunks

The following chunks were changed by the change file: 49, 55, 62, 63, 76, 79, 81, 82, 88, 90.

Index

__DATE__: 82*.
__int128: 28.
__TIME__: 82*.
A: 71.
a: 15 16 27 30 53 60 61 72 78 81∗.
add: 40 75.
addm: 30 38 65 68 70 84 86.
any: 19 40.
argc: 80 81* 82*.
argv: 80 81* 82* 83.
atof: 81*.
atoi: 81* 83.
B: 71.
b: 15 16 30 60 61 72.
best: 53.
both_cost: 52 53 59.
bound: 63∗ 75 76*.
buf: 31.
by: 77.
c: 32 38 53 62∗.
C_CONTINUE: 22 23 26 27.
C_EMPTY: 22 23 26 27.
C_FIRST: 22 23 26 27.
C_LAST: 22 23 26 27.
C_MIDDLE: 22 23 26 27.
C_SINGLETON: 22 23 26 27.
cactus_bound: 57 63*.
calloc: 34 36 81* 84.
canon: 17 20 72 78.
cap: 32 34 36 37.
cells: 72.
cells_v: 69 73 74 76*.
CHECK: 37 79∗ 81* 82*.
ci: 68 69 74 76* 77.
CLD: 49∗ 61 62*.
clock: 68 80 87.
clock_gettime: 68.
CLOCK_MONOTONIC: 68.
CLOCKS_PER_SEC: 68 87.
CLP: 49∗ 61 62*.
CLS: 49∗ 61 62*.
CNT_BITS: 28 82* 83 85.
cnt_t: 28 30 31 32 38 39 40 64 65 66 68 70 71 72 75 81* 82* 84 86.
CNT128: 28.
code: 26 27.
COMPLETE: 18 19 20 75 76*.
completion_possible: 42 63*.
cost: 57 60 61 62*.
count_case: 69 83 84.
cr: 61.
cs: 31 66 68 83 85 86.
cur: 68 69 71 72 74 75 77 78.
CUR_TCH: 49∗ 55* 59.
CUR_V: 49∗ 50 63*.
cycle: 46 54 60 61.
dd: 63∗.
DEAD: 18 19 20 75 76*.
DEDGE: 49∗ 55* 62*.
dist: 63∗.
done: 68.
d1: 62∗.
d2: 62∗.
E: 8.
e: 31 52 53.
eb: 63∗.
empty: 75.
EQ: 64 65 68 70 81* 83 84.
ex: 56 62∗.
exit: 33 36 37 70.
e2: 56 62∗.
f: 14 15 16 17 19 20 72 78 82∗.
fclose: 85.
fflush: 66 68 82*.
finish: 55∗ 56.
first: 26.
first_twin: 20 21 63∗ 73 75 76*.
fopen: 85.
fprintf: 33 36 37 66 68 70 81* 82* 85 87.
free: 34 36.
from: 14.
frow: 50 59 60.
g: 54 60 61.
gap_score: 53 60.
gb: 60.
getbits: 25 27.
gf: 39 40 65 75.
GF: 64 75 78 81*.
h: 34 35 36 51 52 55∗ 56 65 66.
half: 72.
hash: 32 34 36 37.
head: 32 34 36 37 38 39.
height_extra: 56 62*.
hkey: 35 36 37.
hmask: 32 34 36 37.
HO: 49∗ 51 59 62*.
i: 17 19 20 25 26 27 35 36 37 58 59 61 63∗ 72 73 74.
insert_pruned: 40 72 75 78.
is_class: 14 15 16 17.
isc: 72.
i2: 76∗.
j: 14 17 19 26 27 42 50 52 53 60 63∗ 72 75.
jj: 20.
join: 15 16 20.
join_image: 16 20 21.
k: 9 35 61 63∗ 68 69.
key: 13 17 20 26 27 37 40 42 57 58 59 61 63∗ 71 72 75 76*.
keys: 32 34 36 37 75 78.
kink: 57 60 63∗.
KL: 13 17 26 27 37 70 72 75 78.
kmin: 63∗ 70 75 76*.
ktop: 65 66.
KW: 22 26 34 35 36 37 70 75 77 78.
k2: 37 76∗.

L: 8.
l: 26.
la: 76∗.
lab: 19.
last: 20 63∗ 74 75 76* 77.
lastrun: 26 58 60.
last2: 76∗.
lb: 40 63∗ 75 76*.
lb2: 76∗.
lim: 40.
LT: 64 65 68 70 81* 83 84.
m: 17 26 27 42 61 75.
M_u: 8 70 72 78.
M_v: 8 66 70 72.
M_x: 8 66 68 70 83 84.
M_y: 8 66 68 70 83 84.
main: 80.
map: 17.
mask: 72.
MAXBYTES: 64 77 87.
MAXC: 13 20 42 70.
MAXKW: 64 77 87.
MAXL: 8 14 20 26 27 37 49* 58 60 61 69 70 71 72 75 76* 78.
MAXLAB: 14 17 20 72.
MAXRECS: 64 77 87.
MAXSTATES: 64 77 87.
memcmp: 37 78.
memcpy: 20 37.
memset: 17 26 27 36 66 70 72.
MOD: 29 30 81* 82* 83 85.
mp: 20.
mstr: 85.
m1: 54 61.
m1p: 54.
m2: 54 76∗.
n: 32 54 61 66 68 83 84 85 86.
nb: 20 25 60.
nc: 72.
NCL: 49∗ 59 61 62*.
next: 32 38 39.
nh: 72.
NMAX: 8 40 65 66 68 70 72 81* 82* 83 84 85 86.
NOPT: 49∗ 54 56 59.
now: 68.
nrec: 32 34 36 38 68 77.
nt: 60 61 63∗ 75 76*.
ntp: 20.
nt2: 76∗.
nu: 68 69 73 74 76* 77.
nw: 20.
NW: 8 66 70 72 81* 84 85 86.
nxt: 68 69 71 74 75 77 78.
n1: 54.
o: 25 56 84.
occ: 20 75.
OK: 18 19 20 76*.
old: 17.
one: 71 72.
OPTD: 47 49∗ 54 56.
OPTP: 47 49∗ 54 56.
OPTS: 47 49∗ 54 56.
out: 84 85 86.
o2: 76∗.
p: 14 19 31 33 36 37.
pa: 15 16.
pack: 26 37.
par: 14 15 16 17 19 20 72.
pb: 15 16 60.
perror: 85.
PGHOST: 13 14 19 20 26 27 42 61.
pk: 54 60 61.
pos: 25 26 27.
printf: 83 85 86.
PSEC: 67 77 81* 82*.
PSELF: 13 14 15 16 19 27 72.
putbits: 25 26.
p1: 54 61.
p1p: 54.
q: 17 42 82∗.
qb: 60.
R: 58.
r: 31 38 39 52.
ra: 53.
rcap: 32 34 38.
re: 58 60 62*.
*re_*: 53.
reach: 46 51 54 59 61.
realloc: 33.
rec: 32 34 38 39.
rec_t: 32 34 38 82*.
record: 65 75.
RECT: 64 65 66 70 81*.
rect_line: 66 69 70.
REFLECT: 78 79∗ 81* 82*.
relabel: 14 15 16.
RESFILE: 79∗ 81* 82* 85.
retire: 19 20.
rf: 85.
rkey: 78.
rl: 58 60.
rs: 58 60 62*.
s: 30 33 55∗ 56.
sb: 60.
sc: 51 54 60.
scs: 61.
sep: 55∗.
si: 38 39 40 74 75 78.
sk: 20 75 78.
sl: 60.
snprintf: 85.
sp: 27 60.
sreach: 46 51 54 59 61.
stack: 27.
stat_line: 68 69 77.
status: 20 75.
stderr: 33 36 37 66 68 70 81* 82* 87.
stdout: 82*.
step: 20 75 76*.

str: 82*.
strcmp: 81*.
strtoull: 81*.
st2: 76*.
sum: 54.
SO: 49* 51 59 62*.
s1: 54 61.
s1p: 54.
t: 34 36 37 38 39 40 54 61 84.
T_LAST: 67 68 77 80.
T_START: 67 68 80.
tab_add: 38 40.
tab_bytes: 34 68 77.
tab_clear: 36 74 78.
tab_free: 34 69.
tab_get: 37 40.
tab_grow: 36 37.
tab_init: 34 71.
tab_load: 39 75 78.
tab_unload: 39 75 78.
table_t: 32 34 36 37 38 39 40 68 71 77 78.
tag: 68.
tch: 17 20 57 59 60 62* 65 72 75.
tgt: 65.
timespec: 68.
to: 14.
top: 55*.
topgap: 60 61.
toplab: 60 61.
toppk: 60 61.
topsc: 60 61.
tot: 86.
ts: 68.
tv_nsec: 68.
tv_sec: 68.
twice: 84.
ty: 84 85.
tyname: 66 85.
t0: 80 87.
t1: 53.
t2: 53.
uint32_t: 32 34 36 38 39.
uint64_t: 25 26 27 28 29 32 35 37 72.
uint8_t: 14 15 16 17 19 20 26 27 32 37 40 42 57 63* 71 72 75 76* 78.
$u_{\max}$: 8 68 70 83 84.
$u_{\min}$: 8 50 66 70 72 73.
unpack: 27 37 75 78.
use: 78.
v: 68 73.
val: 25 76*.
VERB: 66 67 69 77 81* 82*.
vm: 72.
vspan: 9 65 66 69 70 76*.
VSTAR: 49* 52 55* 63*.
vtop: 66.
v0: 71 72 73.
VO: 49* 63* 72.
w: 25 26 27 37.
wall: 68 77 80.
wt: 32 38 39 40 65.
X: 60.
x: 31 77 78.
xrealloc: 33 34 36 38.

List of Refinements

⟨ Add the self term and the terms for the side and the top 62 ⟩ Used in chunk 57.
⟨ Append the results to the results file 85 ⟩ Used in chunk 80.
⟨ Count a single case and print the result 83 ⟩ Used in chunk 80.
⟨ Count all cases and add them up by type of centre 84 ⟩ Used in chunk 80.
⟨ Create the initial states 72 ⟩ Used in chunk 69.
⟨ Decide cell *ci* of the row for all states 74 ⟩ Used in chunk 69.
⟨ Determine the cells of row *k* 73 ⟩ Used in chunk 69.
⟨ Extend state *si* by the two choices for cell *i* 75 ⟩ Used in chunk 74.
⟨ Find the runs of the frontier 58 ⟩ Used in chunk 57.
⟨ Global variables 8 13 22 29 49 64 67 79 ⟩ Used in chunk 2.
⟨ Include files 3 ⟩ Used in chunk 2.
⟨ Initialise the height computation with the frontier cells 59 ⟩ Used in chunk 57.
⟨ Local variables of *count_case* 71 ⟩ Used in chunk 69.
⟨ Merge every state with its mirror image 78 ⟩ Used in chunk 69.
⟨ Parse the command line 81 ⟩ Used in chunk 80.
⟨ Pay for the outer paths, the rings and the middle 61 ⟩ Used in chunk 57.
⟨ Print the final statistics 87 ⟩ Used in chunk 80.
⟨ Print the program parameters 82 ⟩ Used in chunk 80.
⟨ Print the table of results 86 ⟩ Used in chunk 80.
⟨ Scan the runs from left to right and pay for the pockets 60 ⟩ Used in chunk 57.
⟨ Set up the geometry of the case 70 ⟩ Used in chunk 69.
⟨ Sharpen *lb* by looking one cell ahead 76 ⟩ Used in chunk 75.
⟨ Subroutines 9 14 15 16 17 19 20 25 26 27 30 31 33 34 35 36 37 38 39 40 42 50 51 52 53 54 55 56 57 63 65 66 68 69 ⟩ Used in chunk 2.
⟨ Swap the tables and update the statistics 77 ⟩ Used in chunk 74.
⟨ The main program 80 ⟩ Used in chunk 2.
⟨ Type definitions 18 23 28 32 ⟩ Used in chunk 2.